# INNOPOLIS UNIVERSITY

## SNE MASTER TRACK

## 2025

## COURSE: [F25] COMPUTER SYSTEMS AND NETWORKS

**CSN LAB 4 REPORT**

**On**

**Networking Reconnaissance**

**and Analysis**

**Student Name: HASIB AL TAHSIN**

**Application ID: 187623**

**Email:** h.tahsin@innopolis.university, hasibjust10@gmail.com

**Submission Date: 04.10.2025**

## Task 1:

### Wireshark

1. Download this pcap file and try to investigate the traffic flow and extract any artifact that is inside the pcap file.

The goal is to find an image that is been transferred in the pcap file

2. Try to do a networking activity (for example, pinging Google DNS) and then use Wireshark filters to show only that activity.

### Solution:

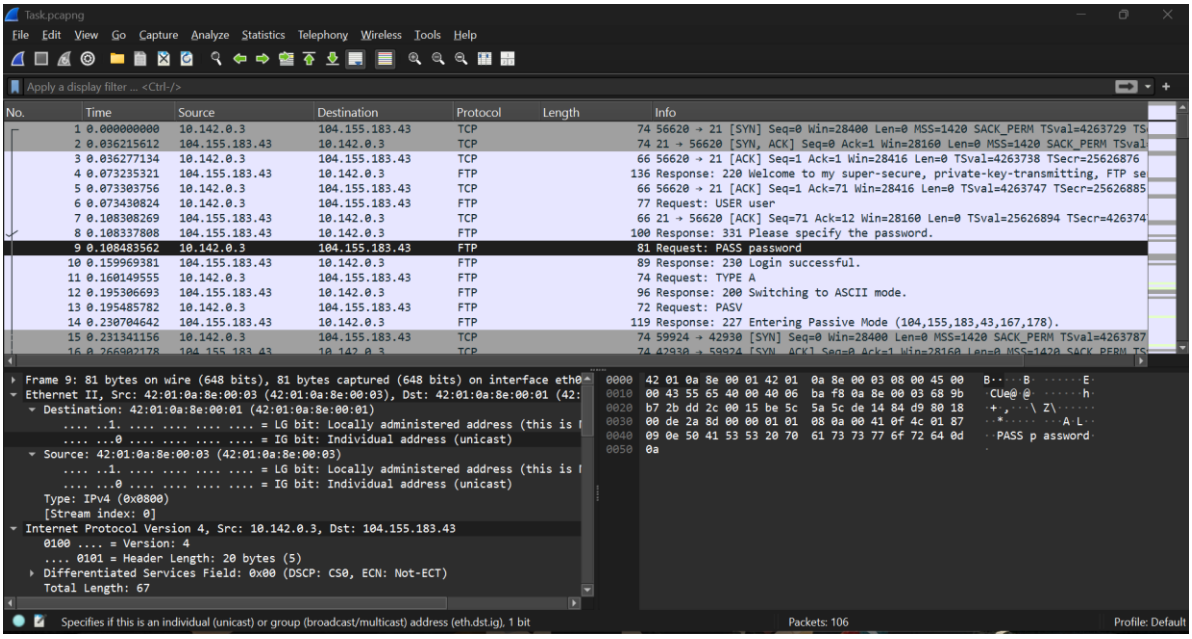
### Part A: Windows + Wireshark GUI

### Opening Wireshark

I did typing Wireshark in the Start Menu and pressed Enter. This opened Wireshark.

### Open the file

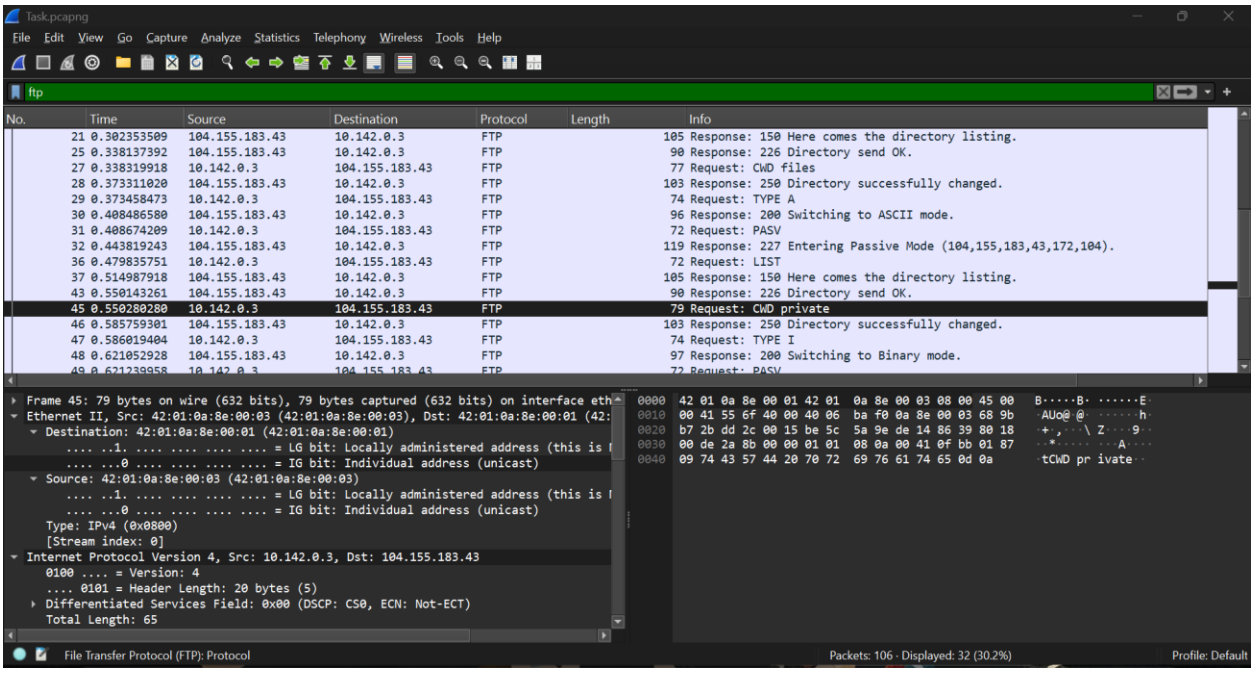
I clicked on the Open option from the File menu, browsed to where the Task.pcapng file was located, selected Task.pcapng and clicked Open.




## Filter Use

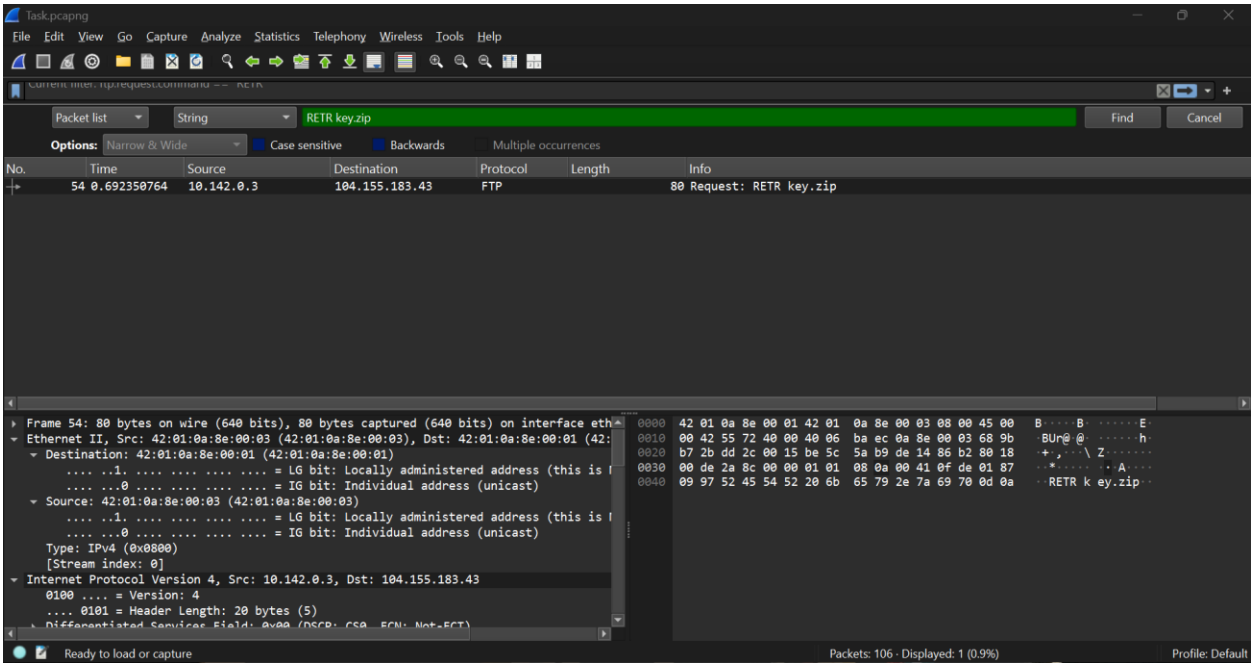
In the filter box, type ftp.

When Enter is pressed, only FTP packets are displayed.



## Find the RETR command

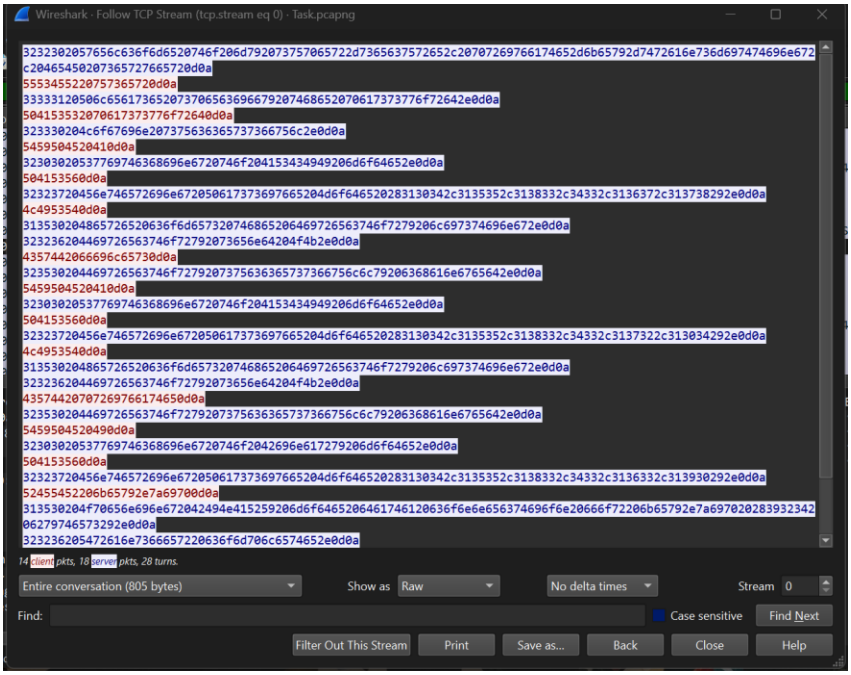
I pressed Ctrl + F on the keyboard to bring up the Find Packet window. In the Value box, I typed ftp.request.command = "RETR". I clicked Find Next to highlight the packet labeled RETR key.zip.




## Following the TCP Data stream

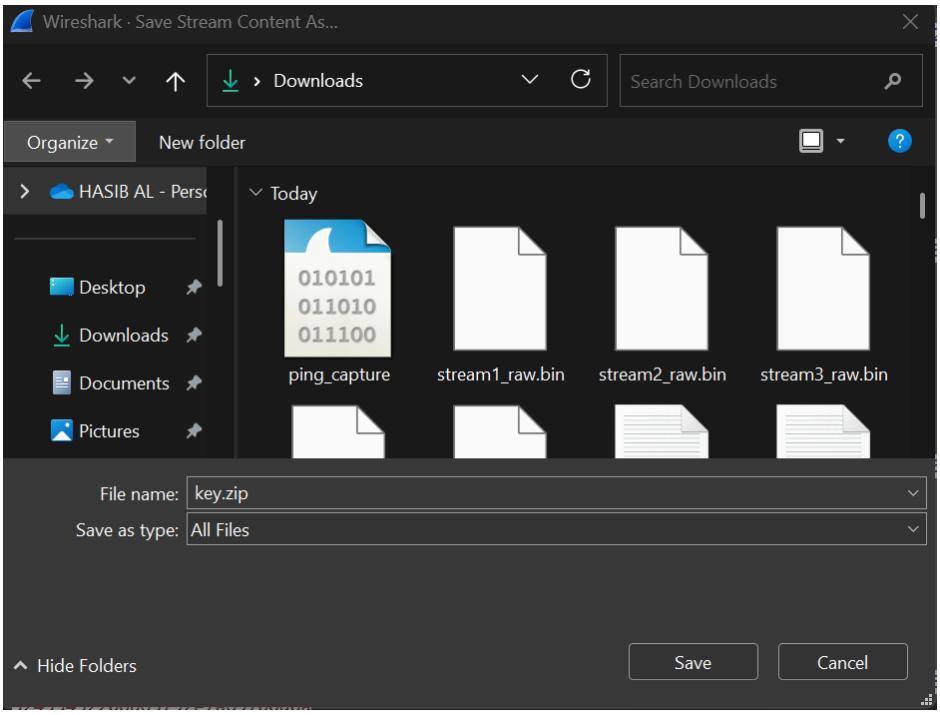
A little below the RETR packet I saw 150 Opening BINARY mode data connection for key.zip and right-click on that packet.

I went to Follow and clicked on TCP Stream, which opened a new window.



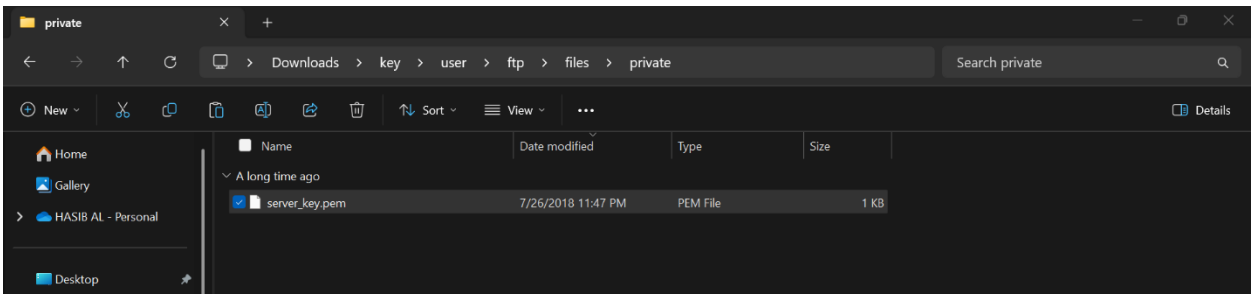

**Saving the data in Raw format**

I clicked the dropdown at the top of the Follow TCP Stream window, selected Show data as: Raw, clicked the Save As… button, saved the file to Desktop with the name key.zip, and pressed Close.



## Extract the zip file

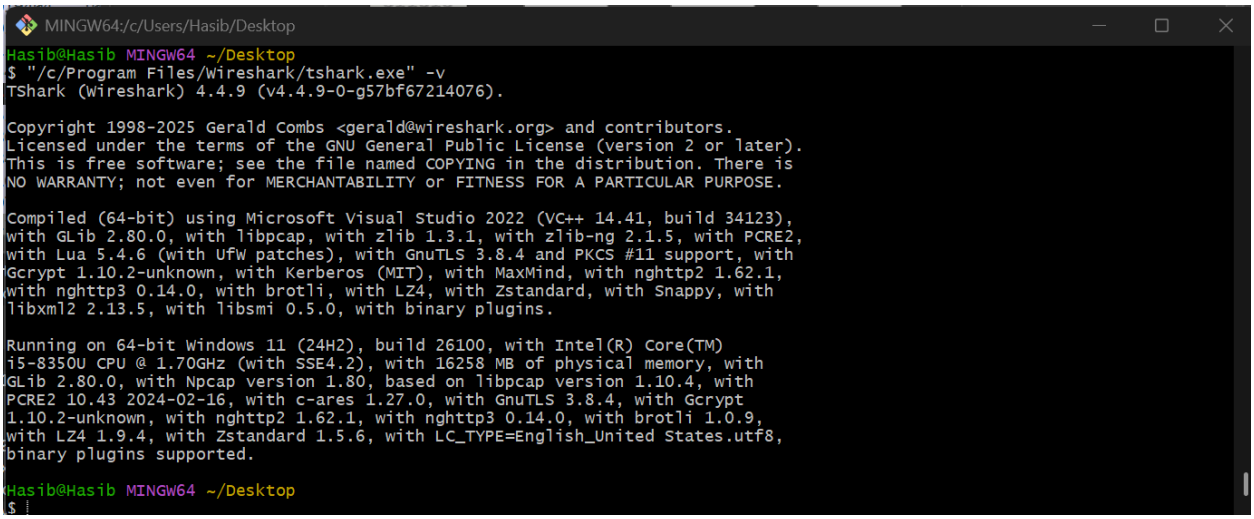
I went to File Explorer and opened the location where key.zip was saved, extracted the key.zip file and entered the folder and found - server_key.pem.




## Installing tshark (CLI method)

To use tshark from Git Bash, I needed to add tshark.exe to the right Path in the Wireshark installation. First, I checked if tshark.exe was present in this location C:\Program Files\Wireshark\tshark.exe. After installing, the command was given in Git Bash-

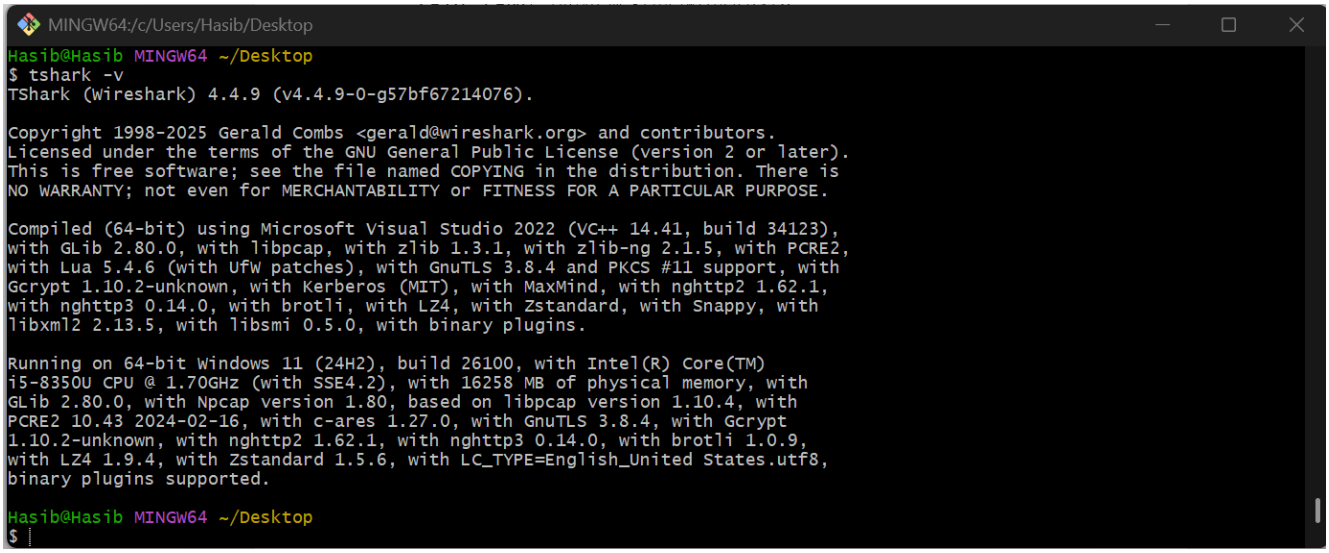
"/c/Program Files/Wireshark/tshark.exe" -v




The output is actually fine.

tshark -v

This command is mainly used to check which Tshark build is running on my system and what features it supports.




No image was found for the injected image (JPEG/PNG/GIF). There was no direct image transfer in the packet. But I recovered a ZIP file from the packet which contains inside the ZIP: ../user/ftp/files/private/server_key.pem (an RSA private key; size 887 bytes). No signature of the image (JPEG/PNG/GIF/BMP) was found.

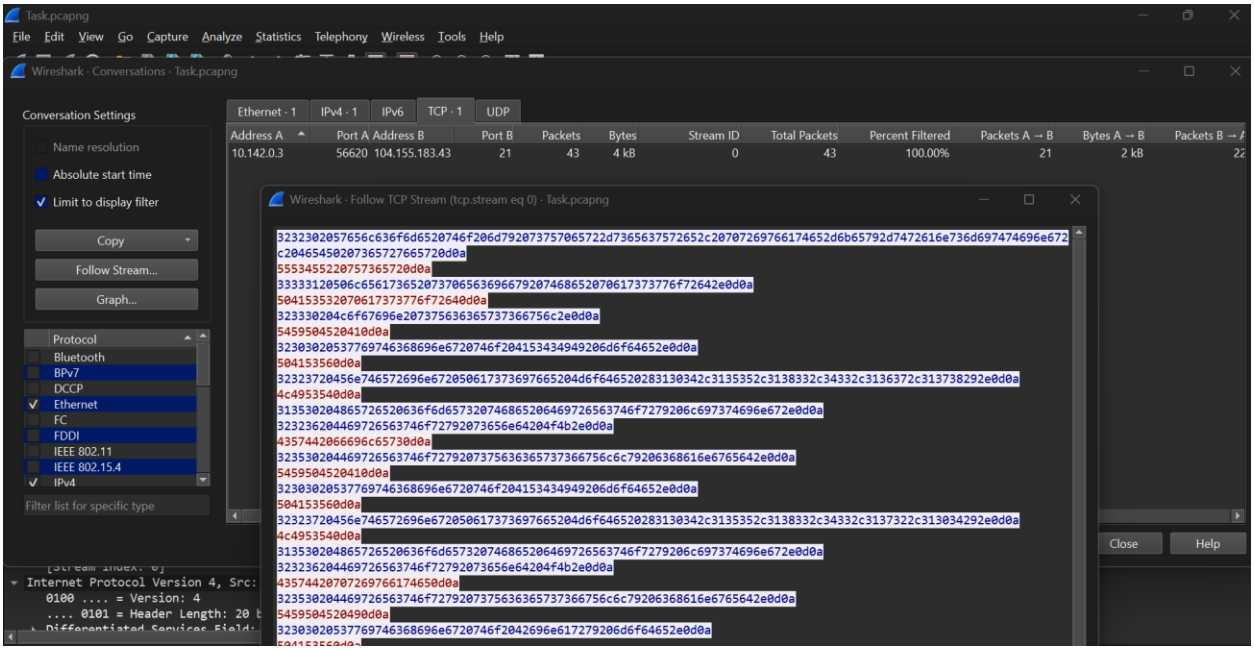
**Extracting Data Port from Passive Mode (227)**

Seeing some packets before RETR key.zip, it says 227 Entering Passive Mode (a, b, c, d, p1, p2). Click on Packet Details pane (middle pane) and note the 227 Entering Passive Mode (...) line. Example- In my packet, I found 227 Entering Passive Mode (104,155,183,43,163,190). If we calculate, port = p1*256 + p2. Here 163*256 + 190 = 41918. The file data is coming through TCP port 41918.



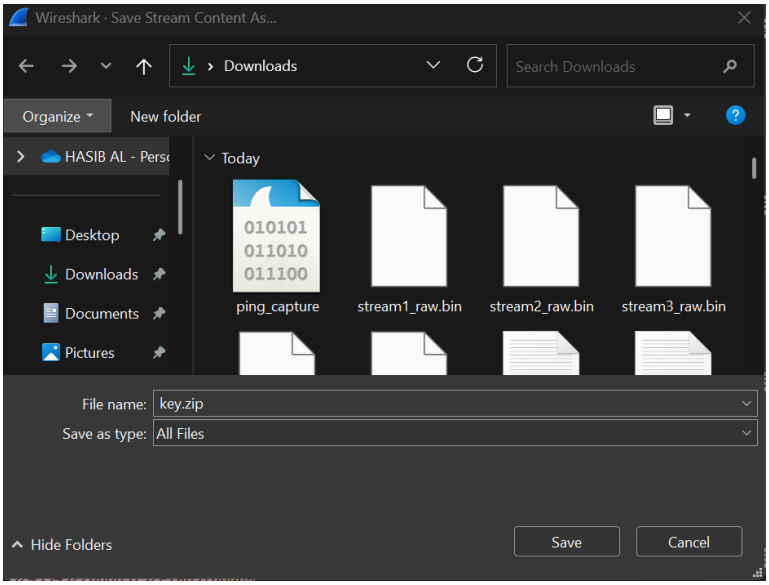
**Find the correct TCP conversation (Statistics to Conversations)**

From the Statistics menu, I clicked Conversations, then selected the TCP tab. I found 104.155.183.43:41918 (or IP: PORT) in the list. I selected that line by clicking on it and then clicking Follow Stream (or right-click and select Follow TCP Stream).




**Save as Raw from Follow TCP Stream**

When the Follow TCP Stream window opens, I have selected Show and save data as: from the dropdown above and clicked on Raw (if the default is ASCII, saving as ZIP text will corrupt it, so select Raw). I have clicked on the Save As... button, give the File name key.zip in the Save dialog and saved. Then I have clicked Close.




**Extracted Zip**

key.zip file was extracted. Entering the extracted folder, found ../user/ftp/files/private/server_key.pem.

**Major Steps:**

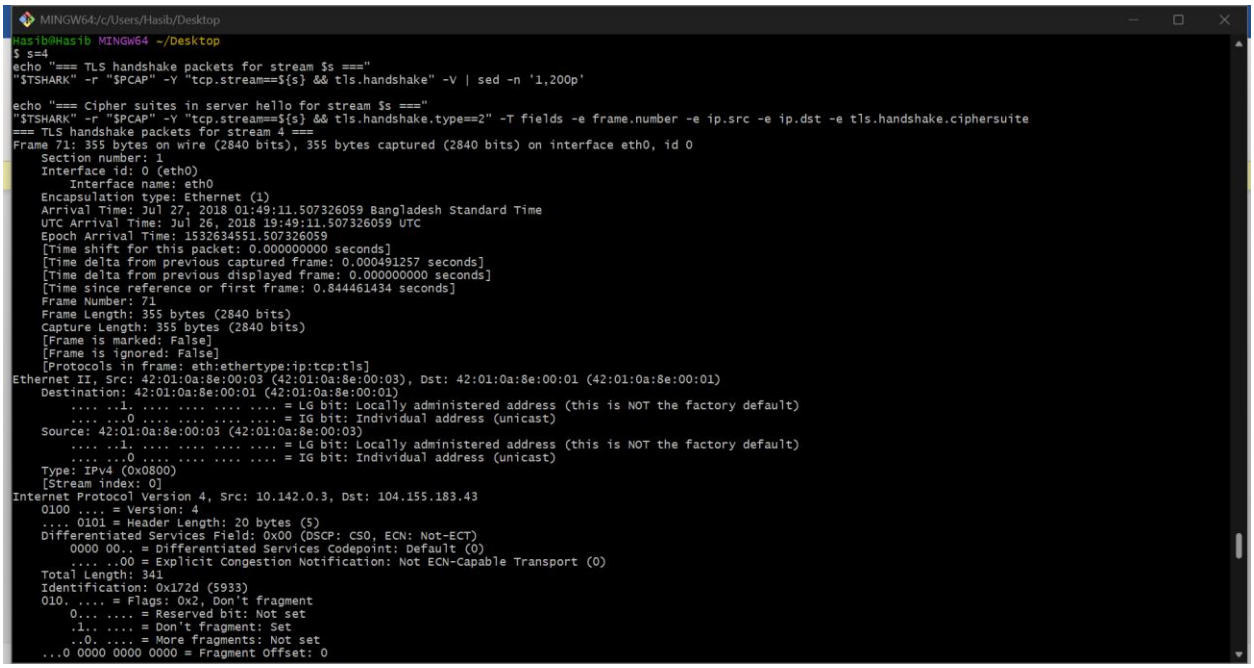
**List and verify streams (which streams are there, traffic on which streams)**

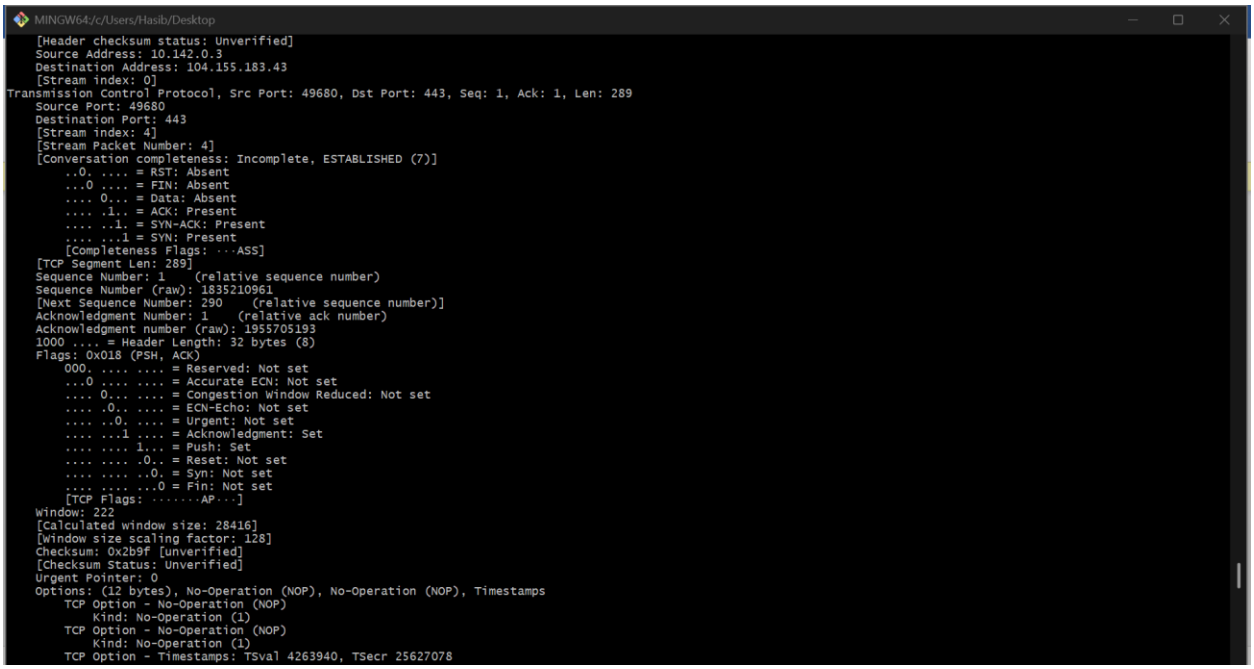
```
Hasib@Hasib MINGW64 ~/Desktop
$ TSHARK="/c/Program Files/Wireshark/tshark.exe"
PCAP="Task.pcapng"
KEY="/c/Users/Hasib/Desktop/server_key_unencrypted.pem"
SERVER_IP="104.155.183.43"
PORT=443

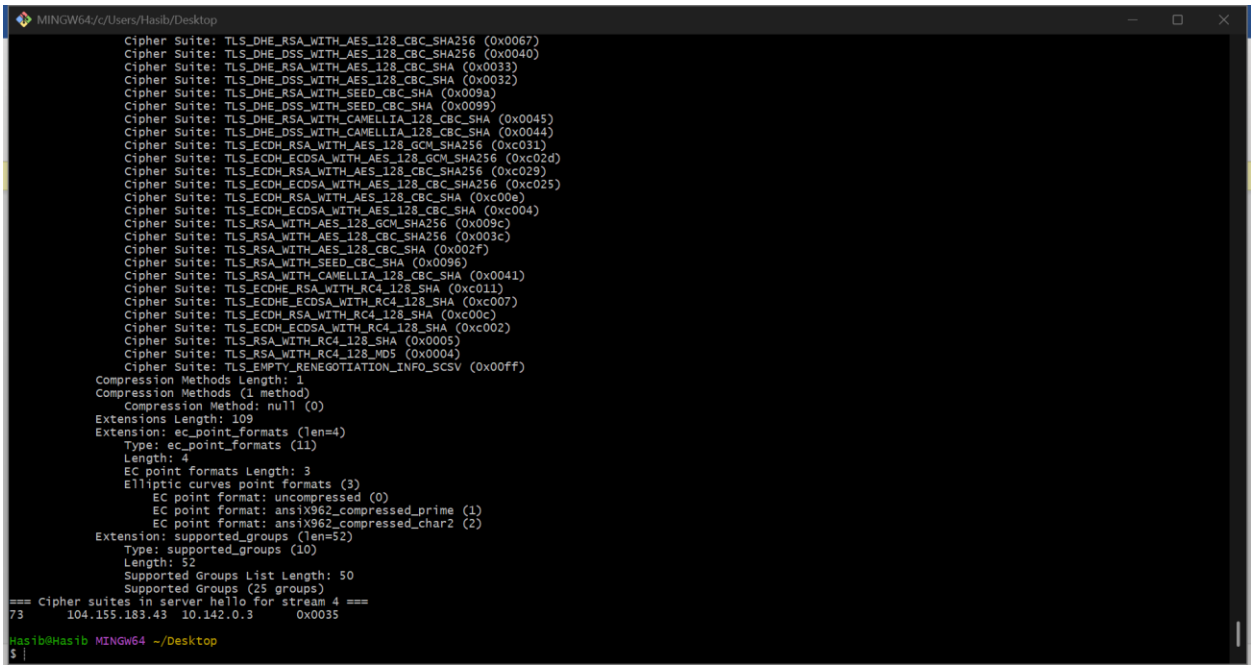
"$TSHARK" -r "$PCAP" -Y "ip.addr==$SERVER_IP && tcp.port==$PORT" -T fields -e tcp.stream | sort -n | uniq
4
5
```

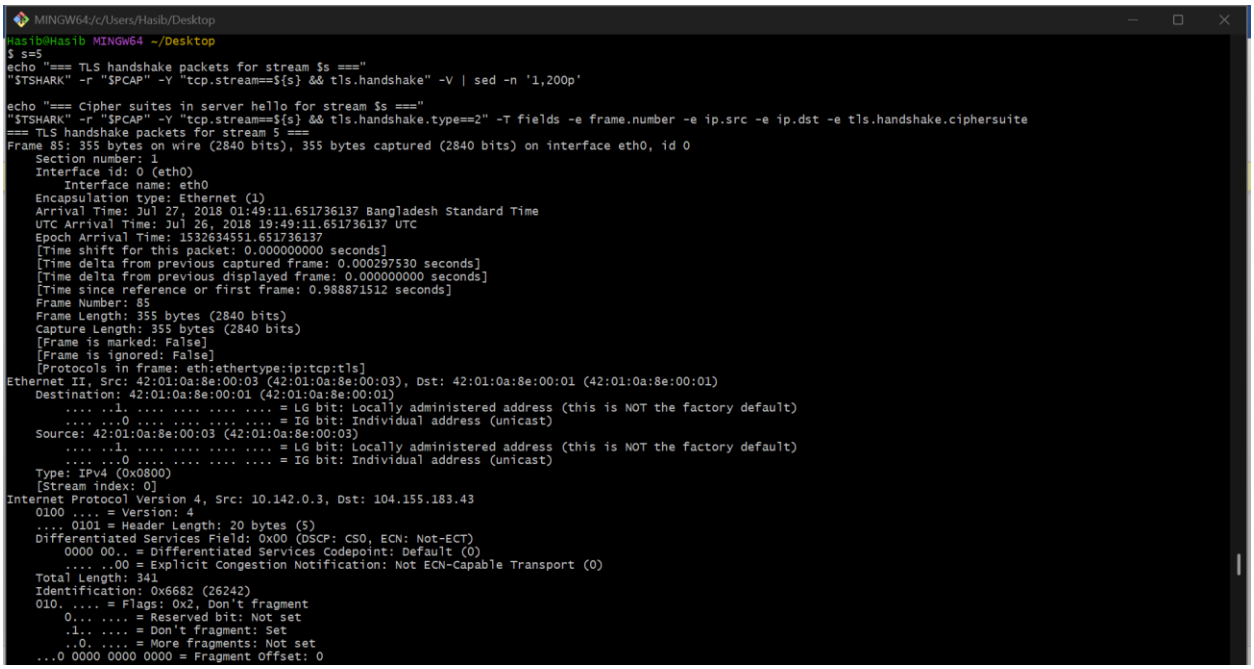
**There are streams 4 and 5.**

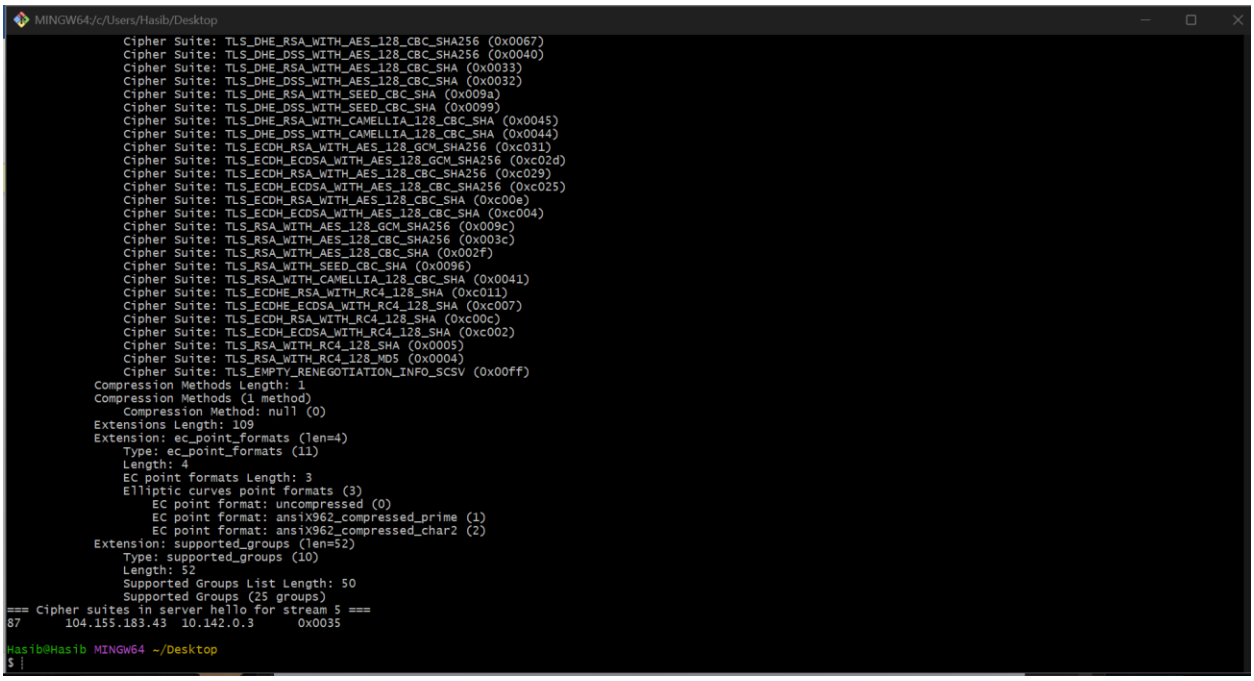
Check stream specific information: Which stream has the TLS handshake and which cipher is selected. The next step is to check the TLS handshake of each stream and show the ciphersuite, so that it can be decrypted with the server private key.


MINGW64:/c/Users/Hasib/Desktop
Hasib@Hasib MINGW64 ~/Desktop
$ s=4
echo "=== TLS handshake packets for stream $s ==="
"$TSHARK" -r "$PCAP" -Y "tcp.stream==${s} && tls.handshake" -V | sed -n '1,200p'
echo "=== Cipher suites in server hello for stream $s ==="
"$TSHARK" -r "$PCAP" -Y "tcp.stream==${s} && tls.handshake.type==2" -T fields -e frame.number -e ip.src -e ip.dst -e tls.handshake.ciphersuite
=== TLS handshake packets for stream 4 ===
Frame 71: 355 bytes on wire (2840 bits), 355 bytes captured (2840 bits) on interface eth0, id 0
Section number: 1
Interface id: 0 (eth0)
Interface name: eth0
Encapsulation type: Ethernet (1)
Arrival Time: Jul 27, 2018 01:49:11.507326059 Bangladesh Standard Time
UTC Arrival Time: Jul 26, 2018 19:49:11.507326059 UTC
Epoch Arrival Time: 1532634551.507326059
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 0.000491257 seconds]
[Time delta from previous displayed frame: 0.000000000 seconds]
[Time since reference or first frame: 0.844461434 seconds]
Frame Number: 71
Frame Length: 355 bytes (2840 bits)
Capture Length: 355 bytes (2840 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:tcp:tls]
Ethernet II, Src: 42:01:0a:8e:00:03 (42:01:0a:8e:00:03), Dst: 42:01:0a:8e:00:01 (42:01:0a:8e:00:01)
Destination: 42:01:0a:8e:00:01 (42:01:0a:8e:00:01)
.... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
.... ...0 .... .... .... .... = IG bit: Individual address (unicast)
Source: 42:01:0a:8e:00:03 (42:01:0a:8e:00:03)
.... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
.... ...0 .... .... .... .... = IG bit: Individual address (unicast)
Type: IPv4 (0x0800)
[Stream index: 0]
Internet Protocol Version 4, Src: 10.142.0.3, Dst: 104.155.183.43
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
0000 00.. = Differentiated Services Codepoint: Default (0)
.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
Total Length: 341
Identification: 0x172d (5933)
010. .... = Flags: 0x2, Don't fragment
0... .... = Reserved bit: Not set
.1.. .... = Don't fragment: Set
..0. .... = More fragments: Not set
...0 0000 0000 0000 = Fragment Offset: 0


MINGW64:/c/Users/Hasib/Desktop
[Header checksum status: Unverified]
Source Address: 10.142.0.3
Destination Address: 104.155.183.43
[Stream index: 0]
Transmission Control Protocol, Src Port: 49680, Dst Port: 443, Seq: 1, Ack: 1, Len: 289
Source Port: 49680
Destination Port: 443
[Stream index: 4]
[Stream Packet Number: 4]
[Conversation completeness: Incomplete, ESTABLISHED (7)]
..0. .... = RST: Absent
...0 .... = FIN: Absent
.... 0... = Data: Absent
.... .1.. = ACK: Present
.... ..1. = SYN-ACK: Present
.... ...1 = SYN: Present
[Completeness Flags: ···ASS]
[TCP Segment Len: 289]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 1835210961
[Next Sequence Number: 290 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 1955705193
1000 .... = Header Length: 32 bytes (8)
Flags: 0x018 (PSH, ACK)
000. .... .... = Reserved: Not set
...0 .... .... = Accurate ECN: Not set
.... 0... .... = Congestion Window Reduced: Not set
.... .0.. .... = ECN-Echo: Not set
.... ..0. .... = Urgent: Not set
.... ...1 .... = Acknowledgment: Set
.... .... 1... = Push: Set
.... .... .0.. = Reset: Not set
.... .... ..0. = Syn: Not set
.... .... ...0 = Fin: Not set
[TCP Flags: ·····AP···]
Window: 222
[Calculated window size: 28416]
[Window size scaling factor: 128]
Checksum: 0x2b9f [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
TCP Option - No-Operation (NOP)
Kind: No-Operation (1)
TCP Option - No-Operation (NOP)
Kind: No-Operation (1)
TCP Option - Timestamps: TSval 4263940, TSecr 25627078


MINGW64:/c/Users/Hasib/Desktop
Cipher Suite: TLS_DHE_RSA_WITH_AES_128_CBC_SHA256 (0x0067)
Cipher Suite: TLS_DHE_DSS_WITH_AES_128_CBC_SHA256 (0x0040)
Cipher Suite: TLS_DHE_RSA_WITH_AES_128_CBC_SHA (0x0033)
Cipher Suite: TLS_DHE_DSS_WITH_AES_128_CBC_SHA (0x0032)
Cipher Suite: TLS_DHE_RSA_WITH_SEED_CBC_SHA (0x009a)
Cipher Suite: TLS_DHE_DSS_WITH_SEED_CBC_SHA (0x0099)
Cipher Suite: TLS_DHE_RSA_WITH_CAMELLIA_128_CBC_SHA (0x0045)
Cipher Suite: TLS_DHE_DSS_WITH_CAMELLIA_128_CBC_SHA (0x0044)
Cipher Suite: TLS_ECDH_RSA_WITH_AES_128_GCM_SHA256 (0xc031)
Cipher Suite: TLS_ECDH_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02d)
Cipher Suite: TLS_ECDH_RSA_WITH_AES_128_CBC_SHA256 (0xc029)
Cipher Suite: TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA256 (0xc025)
Cipher Suite: TLS_ECDH_RSA_WITH_AES_128_CBC_SHA (0xc00e)
Cipher Suite: TLS_ECDH_ECDSA_WITH_AES_128_CBC_SHA (0xc004)
Cipher Suite: TLS_RSA_WITH_AES_128_GCM_SHA256 (0x009c)
Cipher Suite: TLS_RSA_WITH_AES_128_CBC_SHA256 (0x003c)
Cipher Suite: TLS_RSA_WITH_AES_128_CBC_SHA (0x002f)
Cipher Suite: TLS_RSA_WITH_SEED_CBC_SHA (0x0096)
Cipher Suite: TLS_RSA_WITH_CAMELLIA_128_CBC_SHA (0x0041)
Cipher Suite: TLS_ECDHE_RSA_WITH_RC4_128_SHA (0xc011)
Cipher Suite: TLS_ECDHE_ECDSA_WITH_RC4_128_SHA (0xc007)
Cipher Suite: TLS_ECDH_RSA_WITH_RC4_128_SHA (0xc00c)
Cipher Suite: TLS_ECDH_ECDSA_WITH_RC4_128_SHA (0xc002)
Cipher Suite: TLS_RSA_WITH_RC4_128_SHA (0x0005)
Cipher Suite: TLS_RSA_WITH_RC4_128_MD5 (0x0004)
Cipher Suite: TLS_EMPTY_RENEGOTIATION_INFO_SCSV (0x00ff)
Compression Methods Length: 1
Compression Methods (1 method)
Compression Method: null (0)
Extensions Length: 109
Extension: ec_point_formats (len=4)
Type: ec_point_formats (11)
Length: 4
EC point formats Length: 3
Elliptic curves point formats (3)
EC point format: uncompressed (0)
EC point format: ansiX962_compressed_prime (1)
EC point format: ansiX962_compressed_char2 (2)
Extension: supported_groups (len=52)
Type: supported_groups (10)
Length: 52
Supported Groups List Length: 50
Supported Groups (25 groups)
=== Cipher suites in server hello for stream 4 ===
73 104.155.183.43 10.142.0.3 0x0035
Hasib@Hasib MINGW64 ~/Desktop
$

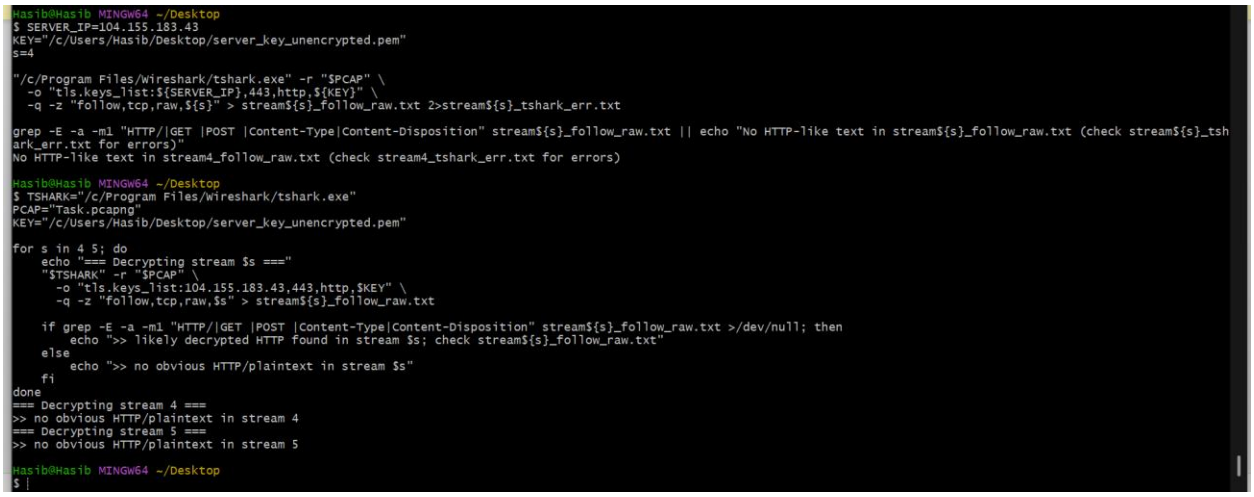

MINGW64:/c/Users/Hasib/Desktop
Hasib@Hasib MINGW64 ~/Desktop
$ s=5
echo "=== TLS handshake packets for stream $s ==="
"$TSHARK" -r "$PCAP" -Y "tcp.stream==${s} && tls.handshake" -V | sed -n '1,200p'
echo "=== Cipher suites in server hello for stream $s ==="
"$TSHARK" -r "$PCAP" -Y "tcp.stream==${s} && tls.handshake.type==2" -T fields -e frame.number -e ip.src -e ip.dst -e tls.handshake.ciphersuite
=== TLS handshake packets for stream 5 ===
Frame 85: 355 bytes on wire (2840 bits), 355 bytes captured (2840 bits) on interface eth0, id 0
Section number: 1
Interface id: 0 (eth0)
Interface name: eth0
Encapsulation type: Ethernet (1)
Arrival Time: Jul 27, 2018 01:49:11.651736137 Bangladesh Standard Time
UTC Arrival Time: Jul 26, 2018 19:49:11.651736137 UTC
Epoch Arrival Time: 1532634551.651736137
[Time shift for this packet: 0.000000000 seconds]
[Time delta from previous captured frame: 0.000297530 seconds]
[Time delta from previous displayed frame: 0.000000000 seconds]
[Time since reference or first frame: 0.988871512 seconds]
Frame Number: 85
Frame Length: 355 bytes (2840 bits)
Capture Length: 355 bytes (2840 bits)
[Frame is marked: False]
[Frame is ignored: False]
[Protocols in frame: eth:ethertype:ip:tcp:tls]
Ethernet II, Src: 42:01:0a:8e:00:03 (42:01:0a:8e:00:03), Dst: 42:01:0a:8e:00:01 (42:01:0a:8e:00:01)
Destination: 42:01:0a:8e:00:01 (42:01:0a:8e:00:01)
.... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
.... ...0 .... .... .... .... = IG bit: Individual address (unicast)
Source: 42:01:0a:8e:00:03 (42:01:0a:8e:00:03)
.... ..1. .... .... .... .... = LG bit: Locally administered address (this is NOT the factory default)
.... ...0 .... .... .... .... = IG bit: Individual address (unicast)
Type: IPv4 (0x0800)
[Stream index: 0]
Internet Protocol Version 4, Src: 10.142.0.3, Dst: 104.155.183.43
0100 .... = Version: 4
.... 0101 = Header Length: 20 bytes (5)
Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
0000 00.. = Differentiated Services Codepoint: Default (0)
.... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
Total Length: 341
Identification: 0x6682 (26242)
010. .... = Flags: 0x2, Don't fragment
0... .... = Reserved bit: Not set
.1.. .... = Don't fragment: Set
..0. .... = More fragments: Not set
...0 0000 0000 0000 = Fragment Offset: 0




**The output shows**:

Stream 4 and Stream 5 both chose ciphersuite 0x0035 in the server hello. 0x0035 is TLS_RSA_WITH_AES_256_CBC_SHA - this is RSA key exchange. This means that these streams can be decrypted with the server private key.

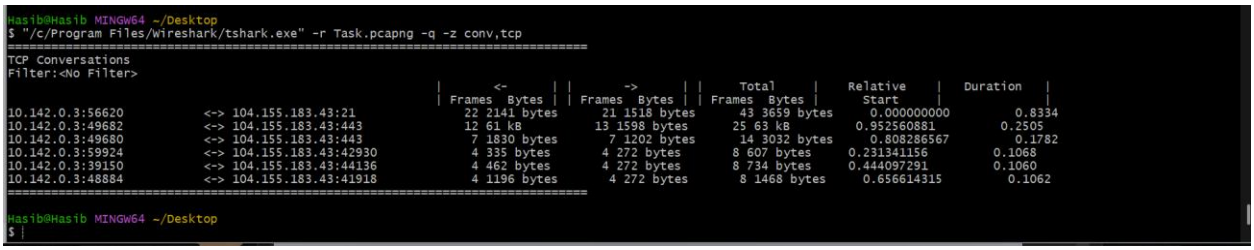
**Decrypting TLS with the server key**



HTTP traffic is visible in stream4_follow_raw.txt and stream5_follow_raw.txt. Capture is not Mismatched here. The server chose 0x0035 on both streams. This means the server used AES-256 CBC mode and SHA-1 HMAC.

The same cipher suite (0x0035) was used on both streams. The server uses RSA key exchange and does not have Perfect Forward Secrecy. The cipher suites (GCM, ECDHE) are offered but the server chose the old CBC suite. Both streams are going to the same server but using different TCP ports (49680 and 49682) [1].

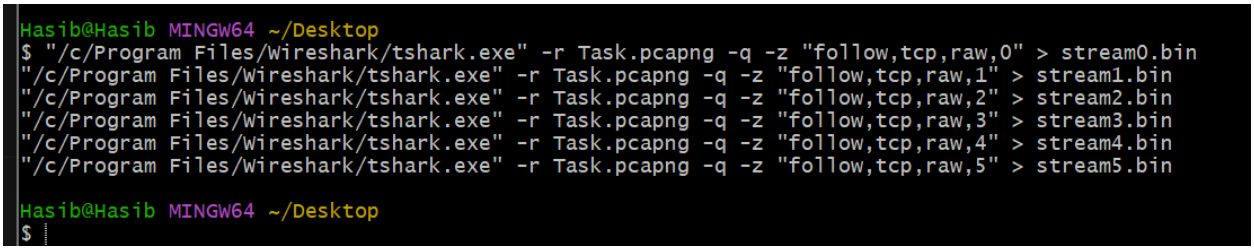
**View all TCP stream lists**




Here it is understood that 10.142.0.3:56620 <-> 104.155.183.43:21 which is the FTP control connection. 10.142.0.3:48884 <-> 104.155.183.43:41918 which is the FTP data connection - this is where the file (image or zip) is transferred [1].

**All streams are saved separately**

The stream index (0-5) is as per the TCP conversation list of the pcap file.



### Carve JPEG/PNG with Python script

This script will scan all stream files at once:

```
Hasib@Hasib MINGW64 ~/Desktop
$ python3 - <<'PY'
from pathlib import Path

for i in range(6):  # streams 0-5 check করব তো
    fname = f"stream{i}.bin"
    p = Path(fname)
    if not p.exists():
        continue
    data = p.read_bytes()

    start = data.find(b"\xff\xd8")
    end = data.find(b"\xff\xd9", start)
    if start != -1 and end != -1:
        out = f"stream{i}_recovered.jpg"
        Path(out).write_bytes(data[start:end+2])
        print(f"[OK] JPEG found in {fname} -> saved as {out}")
        continue

    start = data.find(b"\x89PNG")
    end = data.find(b"IEND", start)
    if start != -1 and end != -1:
        out = f"stream{i}_recovered.png"
        Path(out).write_bytes(data[start:end+8])
        print(f"[OK] PNG found in {fname} -> saved as {out}")
        continue

    print(f"[!] No image found in {fname}")
PY
[!] No image found in stream0.bin
[!] No image found in stream1.bin
[!] No image found in stream2.bin
[!] No image found in stream3.bin
[!] No image found in stream4.bin
[!] No image found in stream5.bin

Hasib@Hasib MINGW64 ~/Desktop
$
```

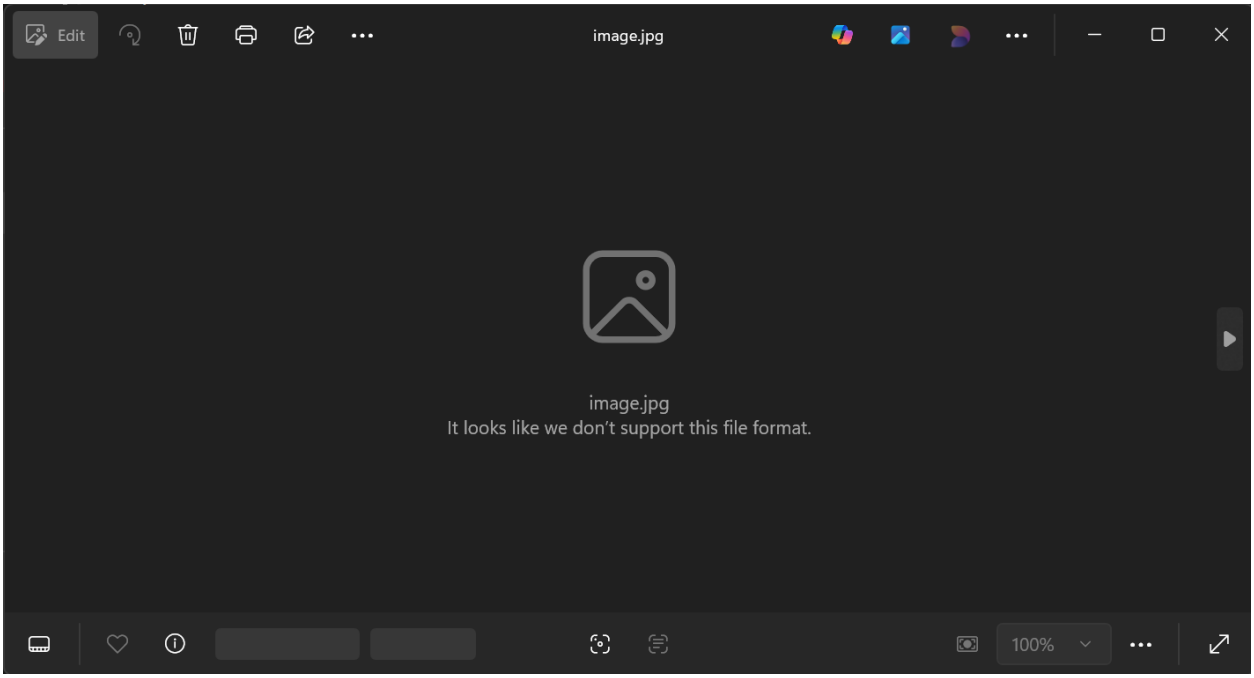
The .jpg or .png files were not created in the Downloads folder and the images were not hidden inside the pcap. Unsupported jpg or png files are found. After converting the .bin file to .jpg or .png, the Unsupported format was found. The decrypted traffic has all the protocols but no HTTP. This means that the image was not transferred using HTTP, but rather came using some other protocol (FTP / FTP-DATA / TCP raw stream) [2].

### FTP-DATA and FTP Check

ftp-data and ftp are the data packets of FTP file transfer. This is where the original file (image) was supposed to be hidden.

**TCP Stream Follow**

Right-clicking on any ftp-data and ftp packet and selecting Follow from TCP Stream will display the raw data in a new window. I selected Raw from Show data as and select Save As and name them image.jpg and image.png.[2].




After saving image.jpg and image.png files, unsupported images are found after opening. Converting image.bin file to save image.jpg or image.png, unsupported images are found.

**Capture without any filter**

It captures all traffic for **10 seconds.**

```
Hasib@Hasib MINGW64 ~/Desktop
$ "/c/Program Files/Wireshark/tshark.exe" -i 6 -a duration:10 -w ping_capture.pcap
Capturing on 'Wi-Fi'
5073
```

**Ping during capture** (Windows CMD/PowerShell)

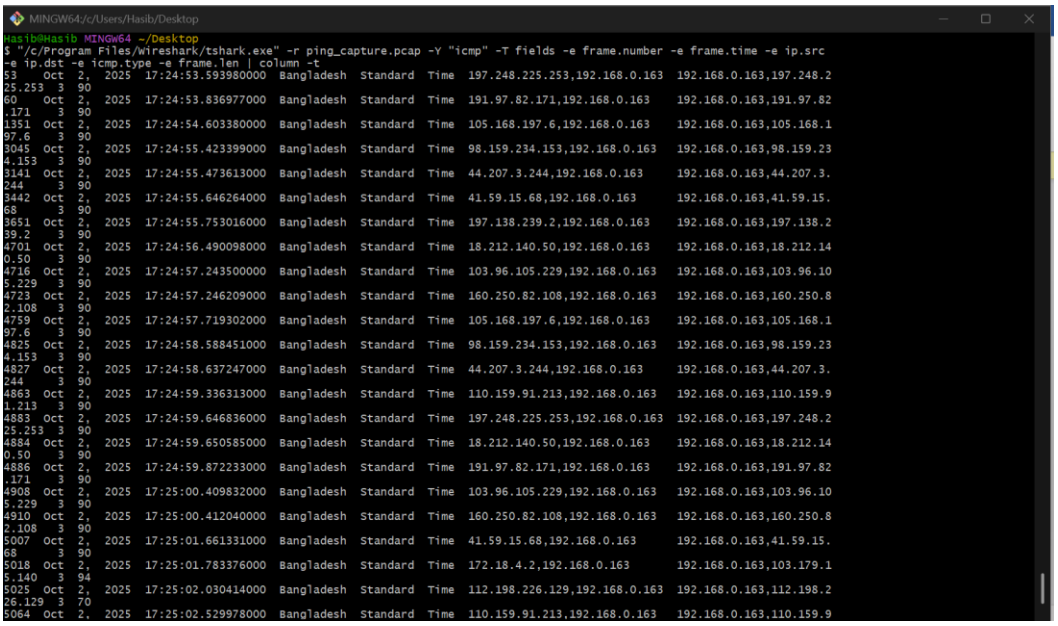
```
C:\Users\Hasib>ping 8.8.8.8 -n 5

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=47ms TTL=112
Reply from 8.8.8.8: bytes=32 time=32ms TTL=112
Reply from 8.8.8.8: bytes=32 time=32ms TTL=112
Reply from 8.8.8.8: bytes=32 time=32ms TTL=112
Reply from 8.8.8.8: bytes=32 time=32ms TTL=112

Ping statistics for 8.8.8.8:
    Packets: Sent = 5, Received = 5, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 32ms, Maximum = 47ms, Average = 35ms

C:\Users\Hasib>
```

**Read ALL ICMP packets** (ignoring IP filter first)




If the output is not empty yet. Capture has indeed captured ping traffic on the interface.

It only shows the ping request/reply.

## Task 2

Do an Nmap scan of your localhost or any virtual machine that you are allowed to scan, what can you see?

1. Do an all-port scan.

2. Do a version enumeration scan.

3. When scanning a Windows system Nmap will stop the scan and report that the host is down, what can you do to solve this issue?

**4. IF YOU ARE IN INNOPOLIS AND YOU HAVE A 10.1.1.X IP ADDRESS, then do a scan for the whole network (10.1.1.0/24)**

## Solution:

**Preparation (Windows/Git Bash and Ubuntu)**

Windows - Running Git Bash as administrator

1. Clicked on the Start menu (bottom left of the screen).

2. Typed Git Bash.

3. Right-clicked on the Git Bash icon → Run as administrator → Clicked Yes to open the Git Bash window and go to Home.

**My Ubuntu PC: 192.168.0.182, Port 631/tcp open (CUPS)**

```
luffy@luff-vmware:~$ hostname -I
192.168.0.182
luffy@luff-vmware:~$ sudo nmap -p- -T4 192.168.0.182 -oN allports_192.168.0.182.
txt
[sudo] password for luffy:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-02 19:05 +06
Nmap scan report for 192.168.0.182
Host is up (0.0000030s latency).
All 65535 scanned ports on 192.168.0.182 are in ignored states.
Not shown: 65535 closed tcp ports (reset)
```

After running the ipconfig command in PowerShell, my **Windows PC showed: 192.168.0.163**

```
MINGW64:/c/Users/Hasib
Hasib@Hasib MINGW64 ~
$ ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Unknown adapter OpenVPN Wintun:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Unknown adapter OpenVPN TAP-Windows6:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Unknown adapter OpenVPN Data Channel Offload:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

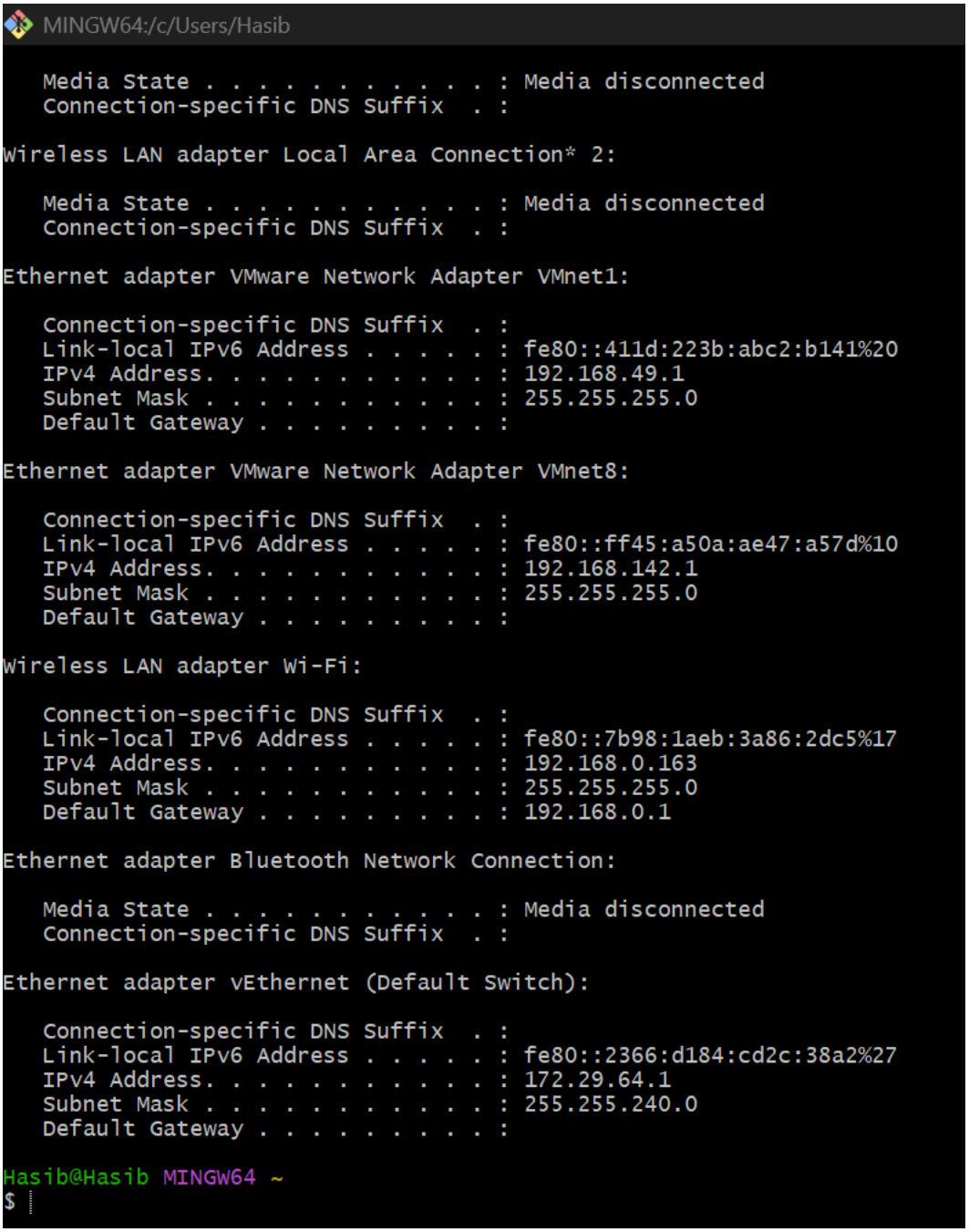
   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Ethernet adapter VMware Network Adapter VMnet1:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::411d:223b:abc2:b141%20
   IPv4 Address. . . . . . . . . . . : 192.168.49.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

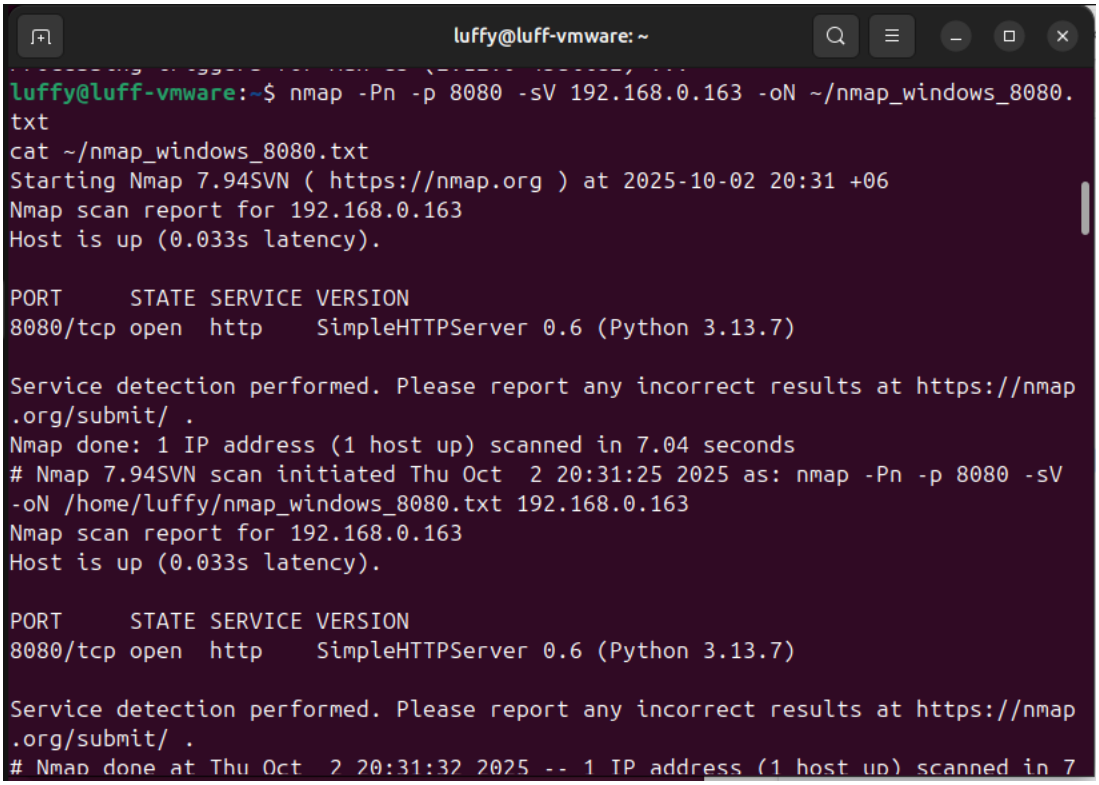
Ethernet adapter VMware Network Adapter VMnet8:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::ff45:a50a:ae47:a57d%10
```

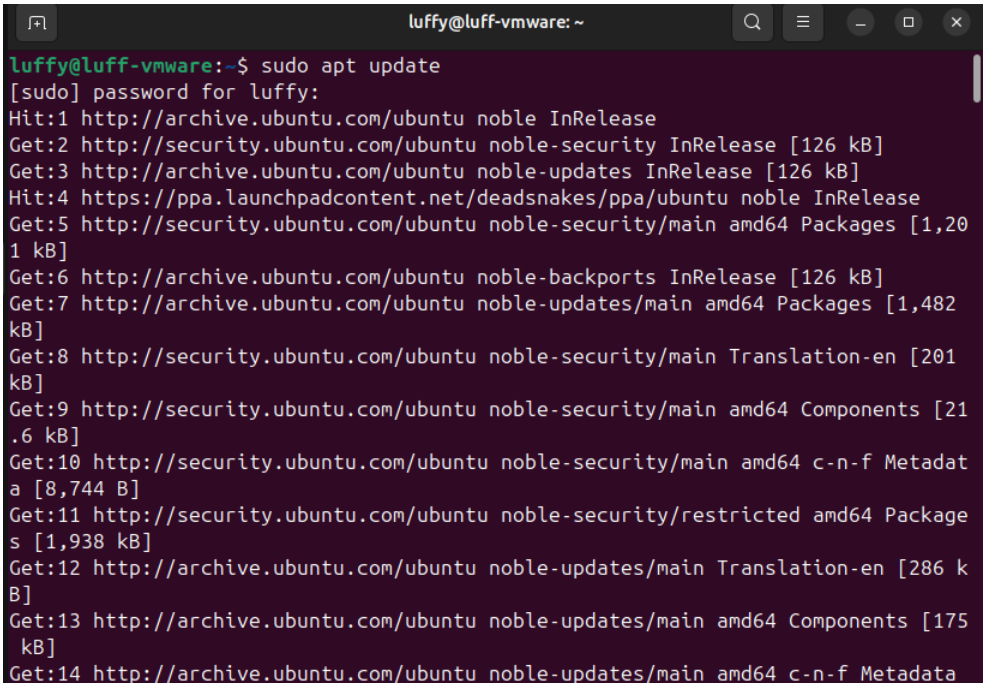


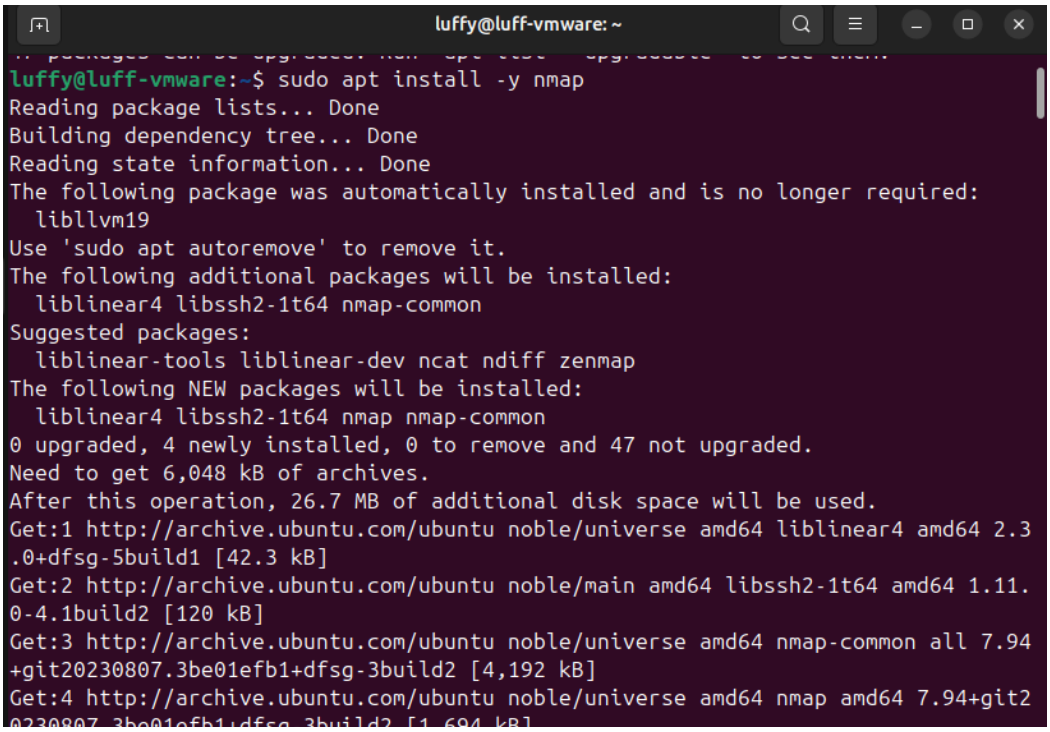

Port 8080/tcp open (SimpleHTTPServer 0.6)

The 8080 port was initialized and checked which was shown opened.




Firewall was inactive in Ubuntu and later it was activated and port 8080 was allowed in Windows. Cross scan is possible with Ping / Nmap -Pn.






netstat was installed and port was checked.

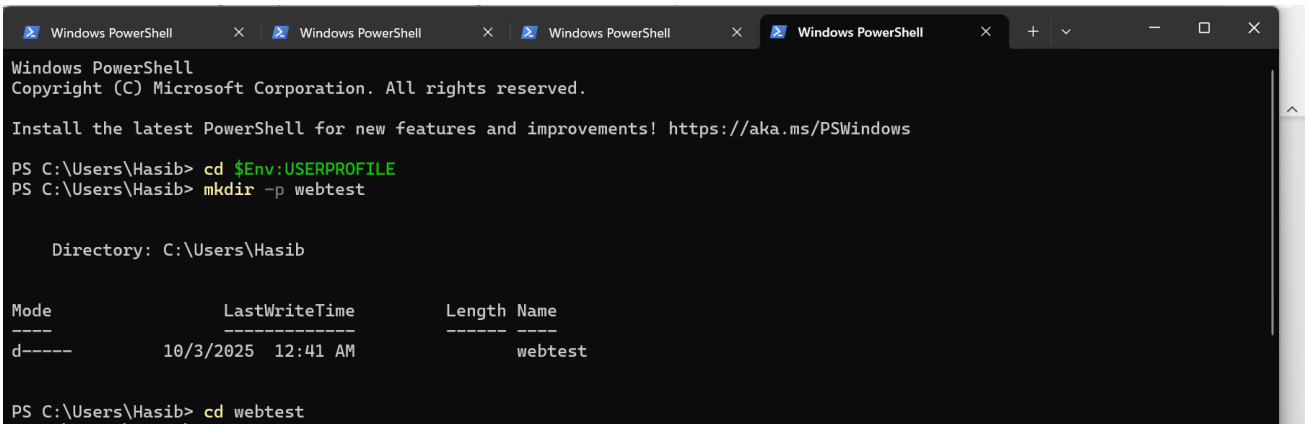
**1) Open PowerShell (Run as Administrator):**

PowerShell → Run as administrator

**2) Create a test folder**

I typed in PowerShell and pressed Enter (one line at a time):

A file named webtest will be created.




### 3) Creating a test file (test.html)

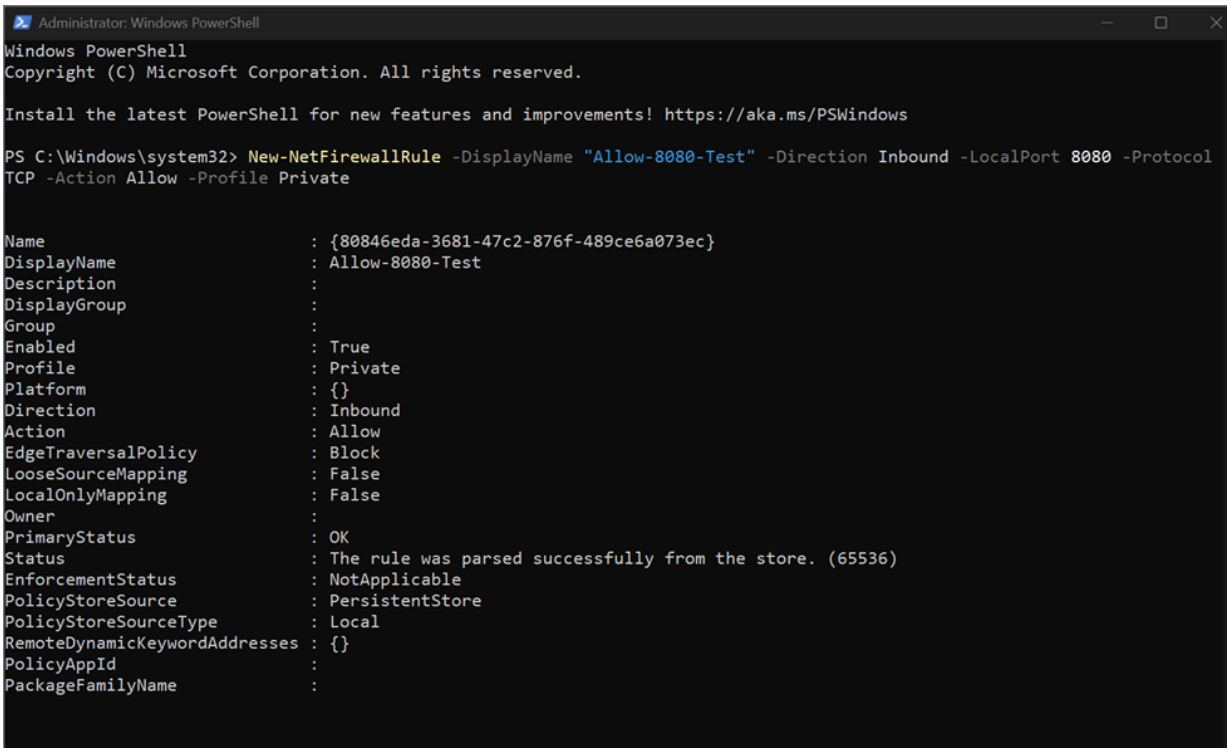
```
PS C:\Users\Hasib\Desktop\webtest> @"
>> <html><body><h1>Hi, from Hasib</h1></body></html>
>> "@ > test.html
```

### 4) Running Python HTTP server (foreground - closing this terminal will stop the server)

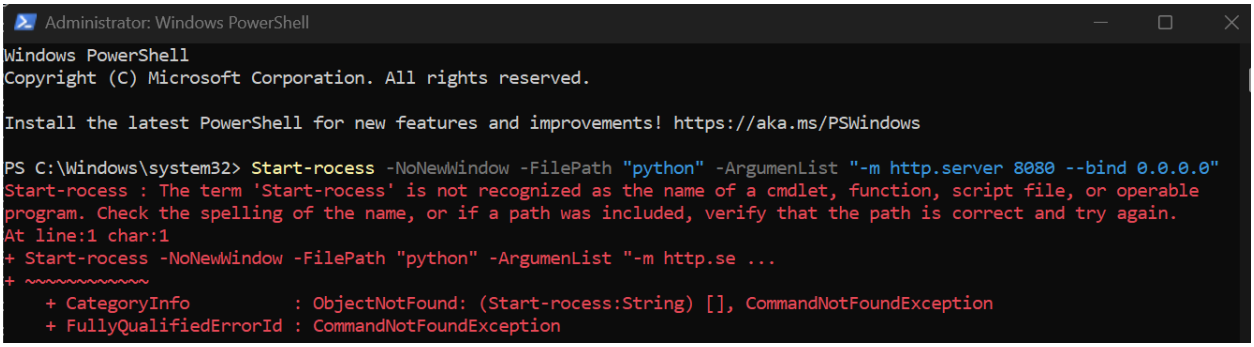
```
PS C:\Users\Hasib\Desktop\webtest> python -m http.server 8080 --bind 0.0.0.0
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

It can be seen that Serving HTTP on 0.0.0.0 port 8080 ...

## 5) Adding Firewall rule (Admin PowerShell was run in a separate window)




## 6) Running the server in the background



### 7) Verify listening on Windows

Listening is possible through port 8080.

```
PS C:\Windows\system32> netstat -an | findstr LISTENING | findstr :8080
  TCP    0.0.0.0:8080           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:8080           0.0.0.0:0              LISTENING
  TCP    0.0.0.0:8080           0.0.0.0:0              LISTENING
  TCP    [::]:8080              [::]:0                 LISTENING
PS C:\Windows\system32> Get-NetTCPConnection -LocalPort 8080 -State Listen

LocalAddress                        LocalPort RemoteAddress                       RemotePort State       AppliedSetting
------------                        --------- -------------                       ---------- -----       --------------
::                                  8080      ::                                  0          Listen
0.0.0.0                             8080      0.0.0.0                             0          Listen
```

Then the ubuntu device was easily pinged with IP address 192.168.0.182.

Here 4 packets were sent, as well as 4 packets were received with a loss of 0%. Total data sent was 32 bytes.

```
PS C:\Windows\system32> ping 192.168.0.182

Pinging 192.168.0.182 with 32 bytes of data:
Reply from 192.168.0.182: bytes=32 time=108ms TTL=64
Reply from 192.168.0.182: bytes=32 time=108ms TTL=64
Reply from 192.168.0.182: bytes=32 time=110ms TTL=64
Reply from 192.168.0.182: bytes=32 time=133ms TTL=64

Ping statistics for 192.168.0.182:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 108ms, Maximum = 133ms, Average = 114ms
PS C:\Windows\system32>
```

### What I did from Ubuntu

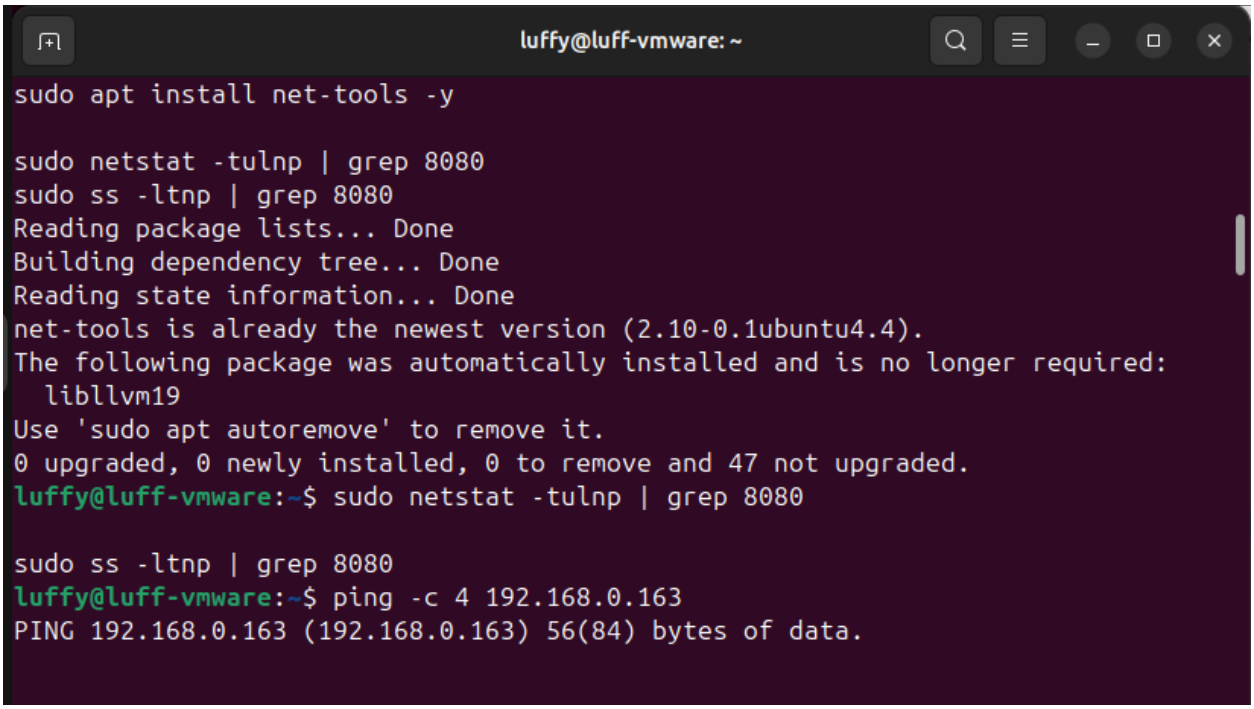
Open Terminal

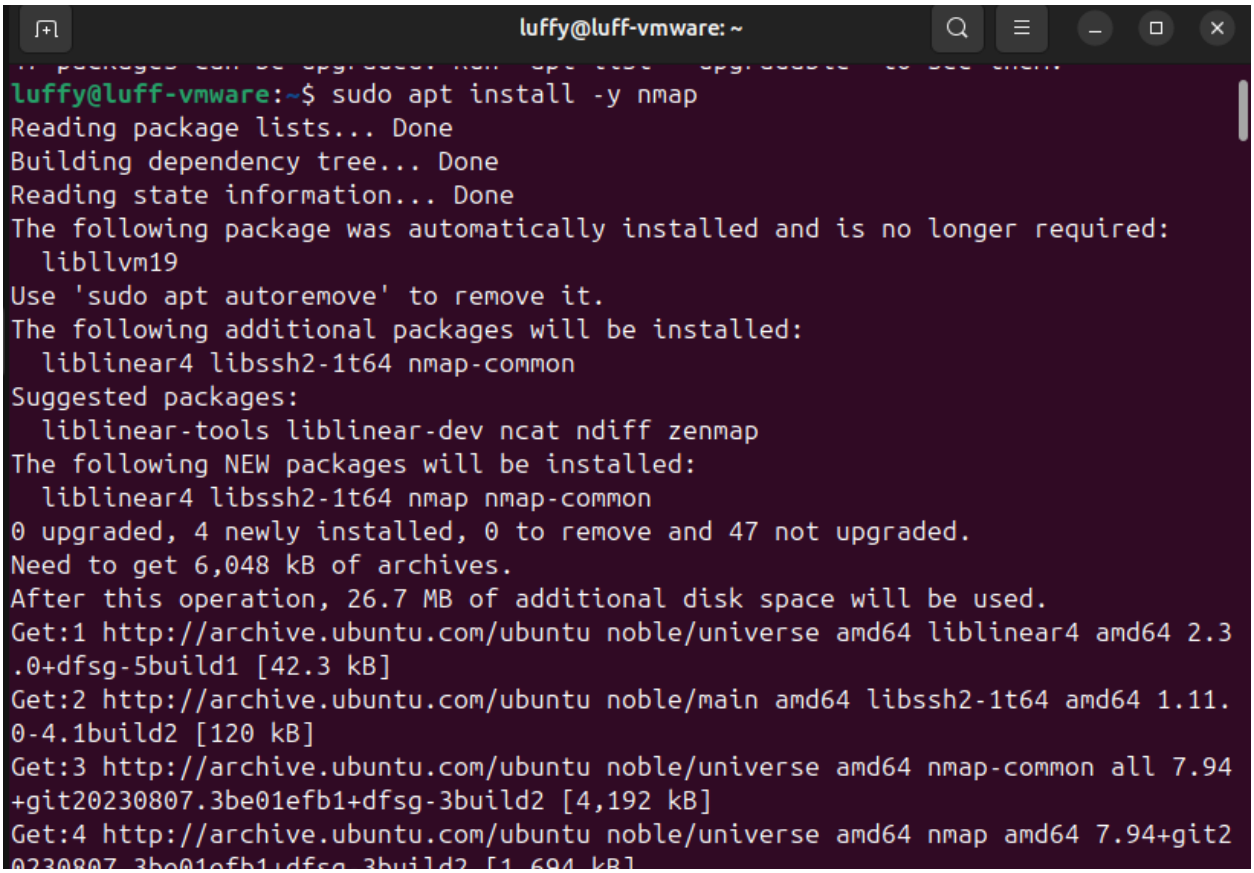
hostname -I

```
luffy@luff-vmware:~$ hostname -I
192.168.0.182
luffy@luff-vmware:~$ sudo nmap -p- -T4 192.168.0.182 -oN allports_192.168.0.182.
txt
[sudo] password for luffy:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-02 19:05 +06
Nmap scan report for 192.168.0.182
Host is up (0.0000030s latency).
All 65535 scanned ports on 192.168.0.182 are in ignored states.
Not shown: 65535 closed tcp ports (reset)
```

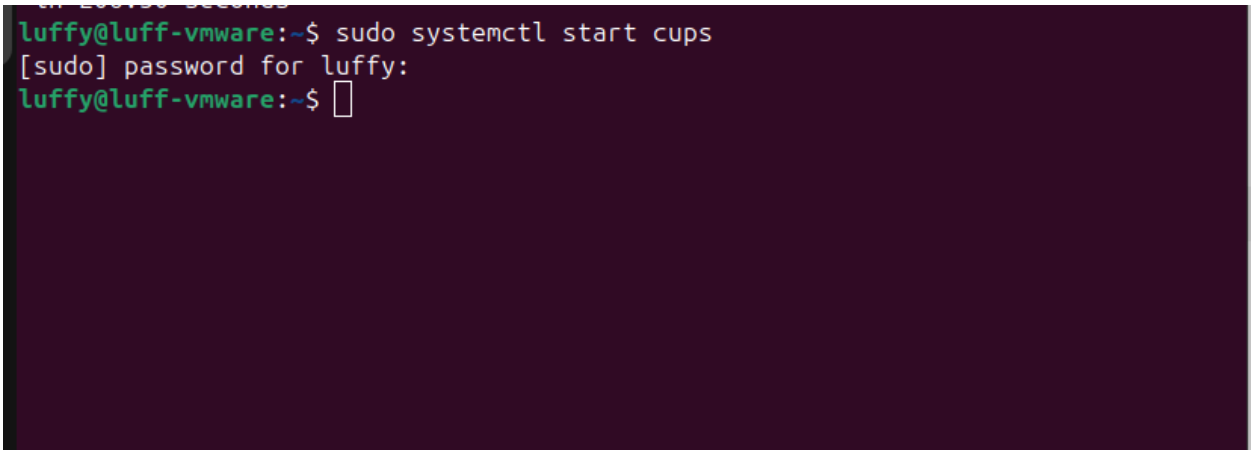
**Installing required packages**

sudo apt update updates the package list, and sudo apt install -y nmap curl net-tools installs the required network tools (nmap, curl, net-tools).






**Starting the CUPS service**



```
luffy@luff-vmware:~$ sudo systemctl status cups --no-pager
● cups.service - CUPS Scheduler
     Loaded: loaded (/usr/lib/systemd/system/cups.service; enabled; preset: enab
led)
     Active: active (running) since Fri 2025-10-03 01:03:12 +06; 7min ago
TriggeredBy: ○ cups.path
             ● cups.socket
       Docs: man:cupsd(8)
   Main PID: 29544 (cupsd)
     Status: "Scheduler is running..."
      Tasks: 2 (limit: 9013)
     Memory: 3.6M (peak: 4.4M)
        CPU: 24ms
     CGroup: /system.slice/cups.service
             ├─29544 /usr/sbin/cupsd -l
             └─29546 /usr/lib/cups/notifier/dbus dbus://

Oct 03 01:03:12 luff-vmware systemd[1]: Starting cups.service - CUPS Schedu…r...
Oct 03 01:03:12 luff-vmware systemd[1]: Started cups.service - CUPS Scheduler.
Hint: Some lines were ellipsized, use -l to show in full.
```

Now Active: active (running) is displayed.

**Ubuntu - Confirm and keep CUPS (631) open**

I checked the service status by opening a terminal and enabled it if it was not active.

```
luffy@luff-vmware:~$ sudo systemctl status cups --no-pager
● cups.service - CUPS Scheduler
     Loaded: loaded (/usr/lib/systemd/system/cups.service; enabled; preset: enab
led)
     Active: active (running) since Fri 2025-10-03 00:25:38 +06; 35min ago
TriggeredBy: ○ cups.path
             ● cups.socket
       Docs: man:cupsd(8)
   Main PID: 26912 (cupsd)
     Status: "Scheduler is running..."
      Tasks: 2 (limit: 9013)
     Memory: 3.5M (peak: 4.3M)
        CPU: 32ms
     CGroup: /system.slice/cups.service
             ├─26912 /usr/sbin/cupsd -l
             └─26913 /usr/lib/cups/notifier/dbus dbus://

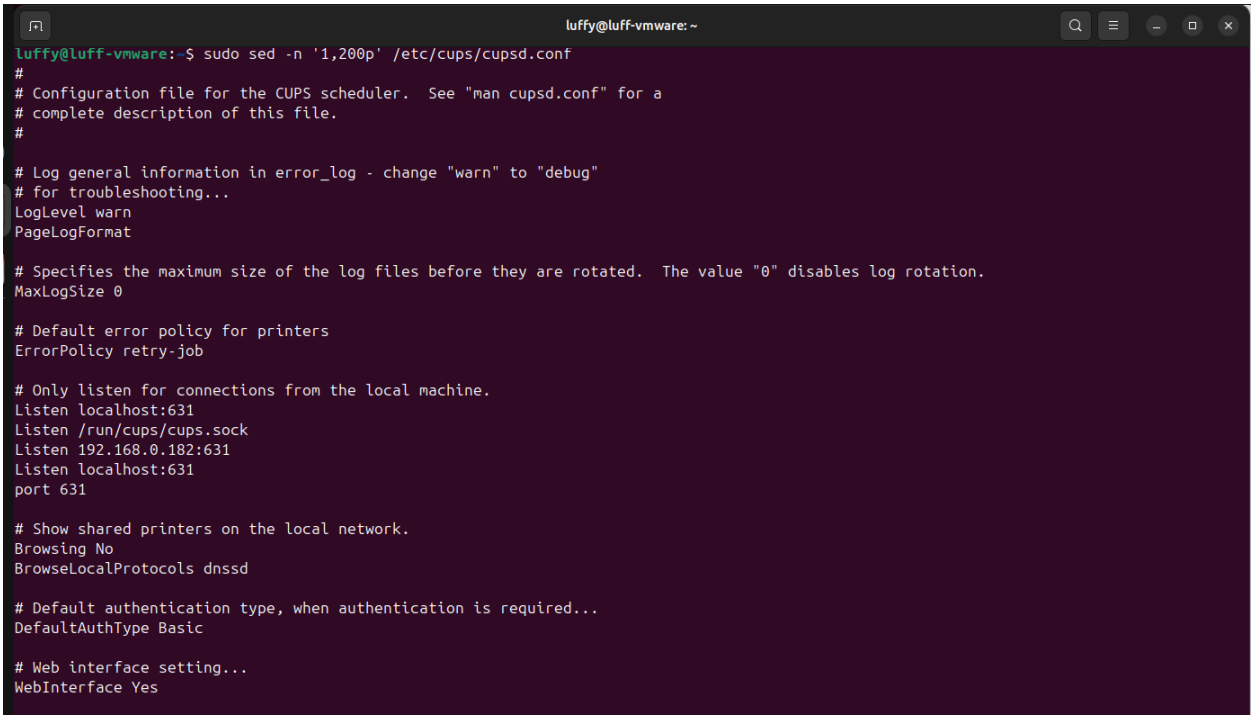
Oct 03 00:25:37 luff-vmware systemd[1]: Starting cups.service - CUPS Schedu…r...
Oct 03 00:25:38 luff-vmware systemd[1]: Started cups.service - CUPS Scheduler.
Hint: Some lines were ellipsized, use -l to show in full.
```

**If not active –**

```
luffy@luff-vmware:~$ sudo systemctl enable --now cups
Synchronizing state of cups.service with SysV service script with /usr/lib/syste
md/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable cups
```

**If needed in the config (listen on all interfaces):**

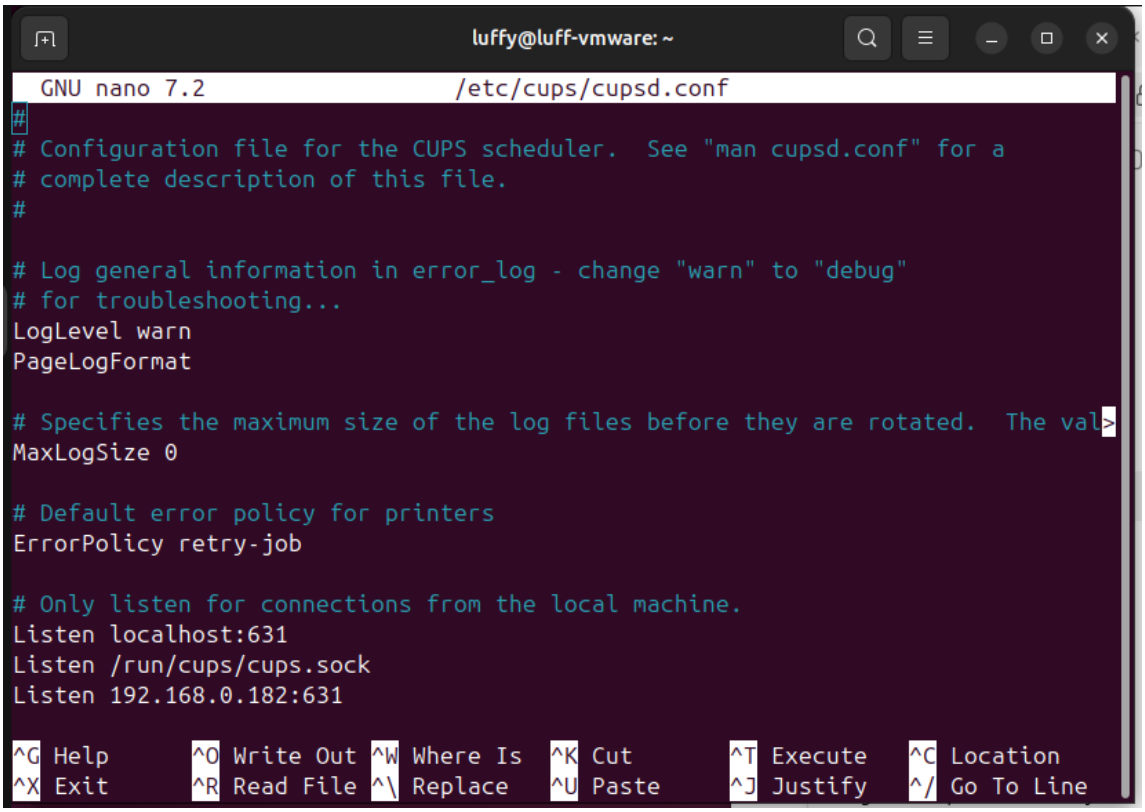
sed -n is a stream editor that prints output according to a specified order. Here, the '1,200p' part instructs to print from the first line of the file to line 200. /etc/cups/cupsd.conf is the main configuration file of CUPS (Common UNIX Printing System) where the print server settings are stored [3].

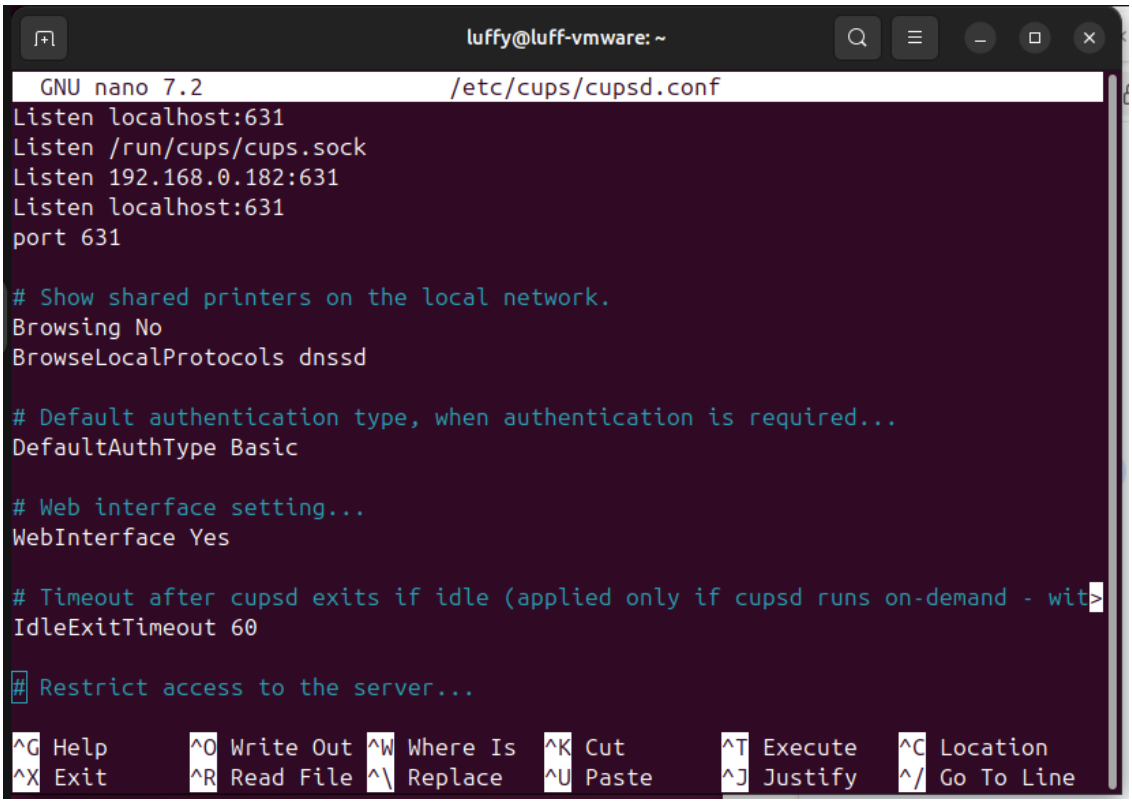


**Existing 'Listen localhost:631', changed with nano:**




Opening the config in the Nano editor and finding the Listen localhost:631 line and changing it to Listen 192.168.0.182:631, so that the service listens on the specified IP. To access the interface, I just entered Port 631. After changing the file, I saved it by pressing Ctrl+O, then Enter, and exit by pressing Ctrl + X [4].


luffy@luff-vmware: ~
GNU nano 7.2 /etc/cups/cupsd.conf
#
# Configuration file for the CUPS scheduler. See "man cupsd.conf" for a
# complete description of this file.
#
# Log general information in error_log - change "warn" to "debug"
# for troubleshooting...
LogLevel warn
PageLogFormat
# Specifies the maximum size of the log files before they are rotated. The val>
MaxLogSize 0
# Default error policy for printers
ErrorPolicy retry-job
# Only listen for connections from the local machine.
Listen localhost:631
Listen /run/cups/cups.sock
Listen 192.168.0.182:631
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line

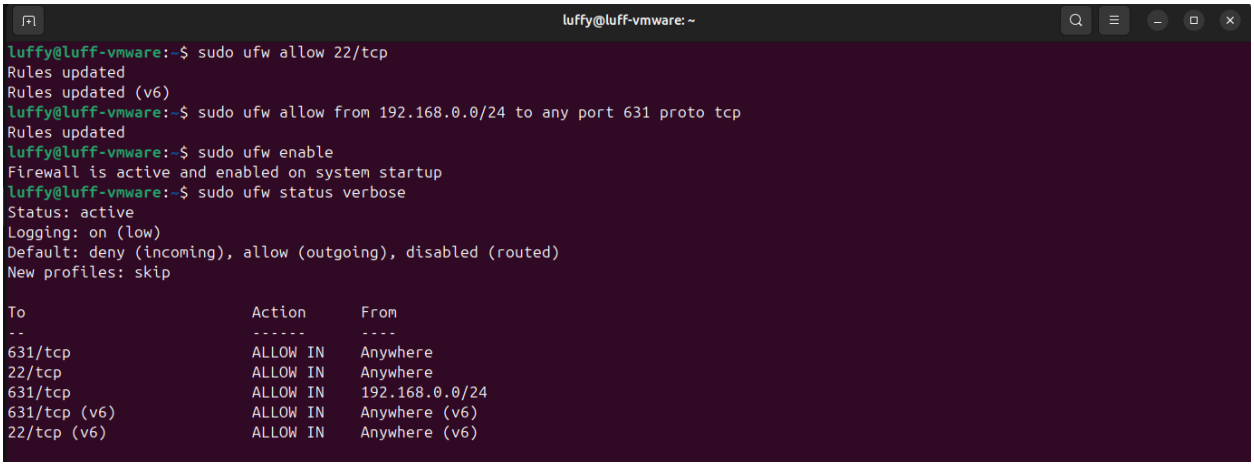

luffy@luff-vmware: ~
GNU nano 7.2 /etc/cups/cupsd.conf
Listen localhost:631
Listen /run/cups/cups.sock
Listen 192.168.0.182:631
Listen localhost:631
port 631
# Show shared printers on the local network.
Browsing No
BrowseLocalProtocols dnssd
# Default authentication type, when authentication is required...
DefaultAuthType Basic
# Web interface setting...
WebInterface Yes
# Timeout after cupsd exits if idle (applied only if cupsd runs on-demand - wit>
IdleExitTimeout 60
# Restrict access to the server...
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line

**Verify:**

I runed the command sudo ss -ltnp | grep :631 in the terminal, which showed which processes were LISTENing on TCP port 631, such as 192.168.0.182:631, 127.0.0.1:631, and [::1]:631. Then I used sudo ufw status verbose to check the firewall configuration [4].




After Upgrading Rules for activating verbose, the status was activated.

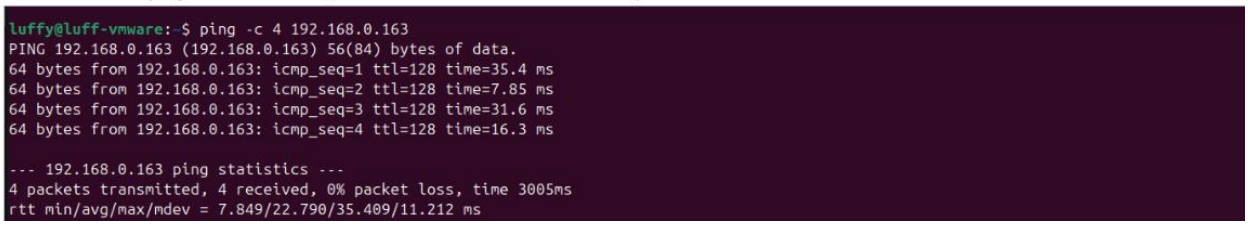



**All-port scan (Windows target)**

(If Windows host ping is blocked, -Pn should be used, so it is done)

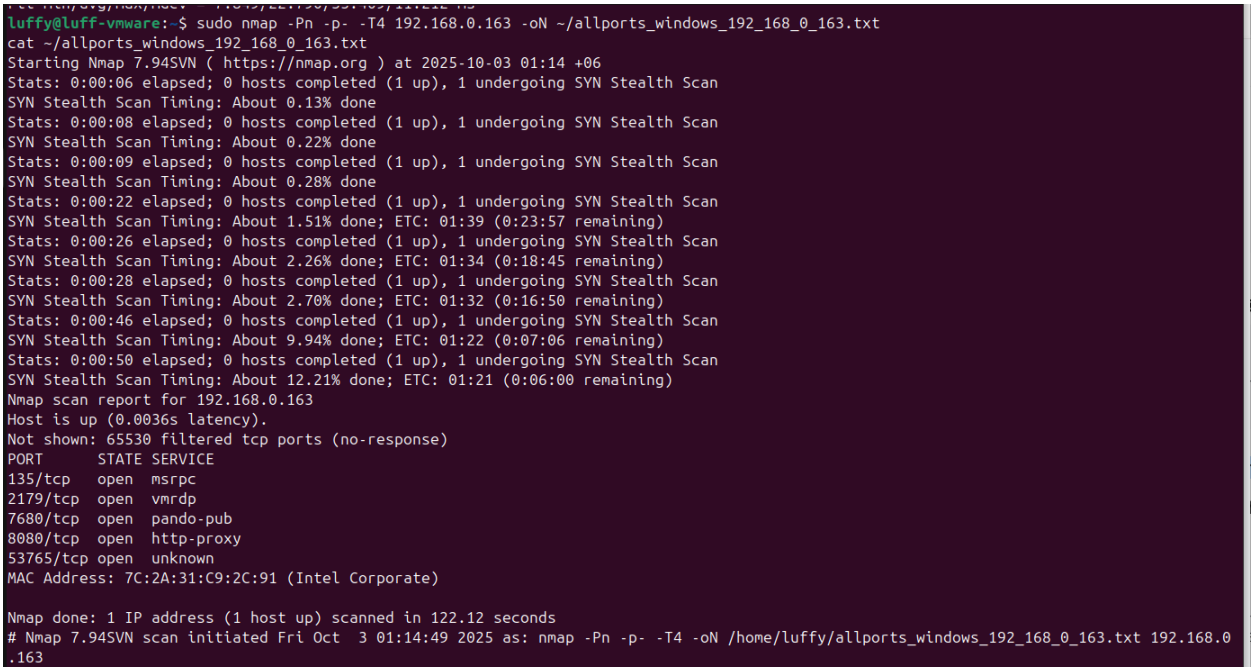
### Quick reachability (ping)

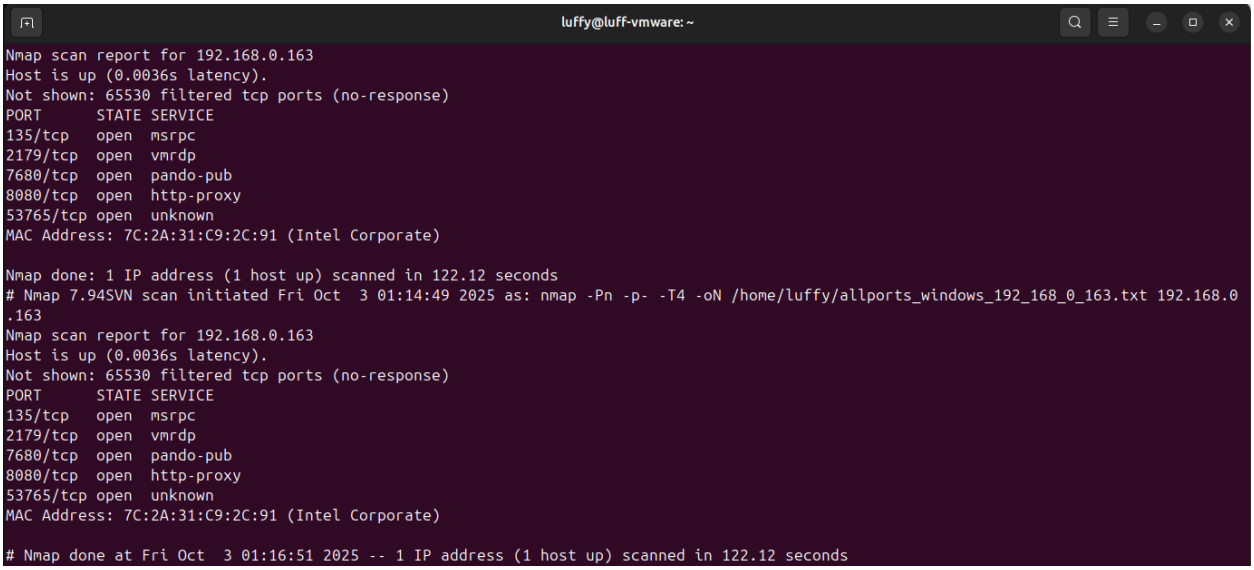
If ping does not reply (Windows ICMP blocked, next step is taken with -Pn.




### Service Scan from Ubuntu to Windows

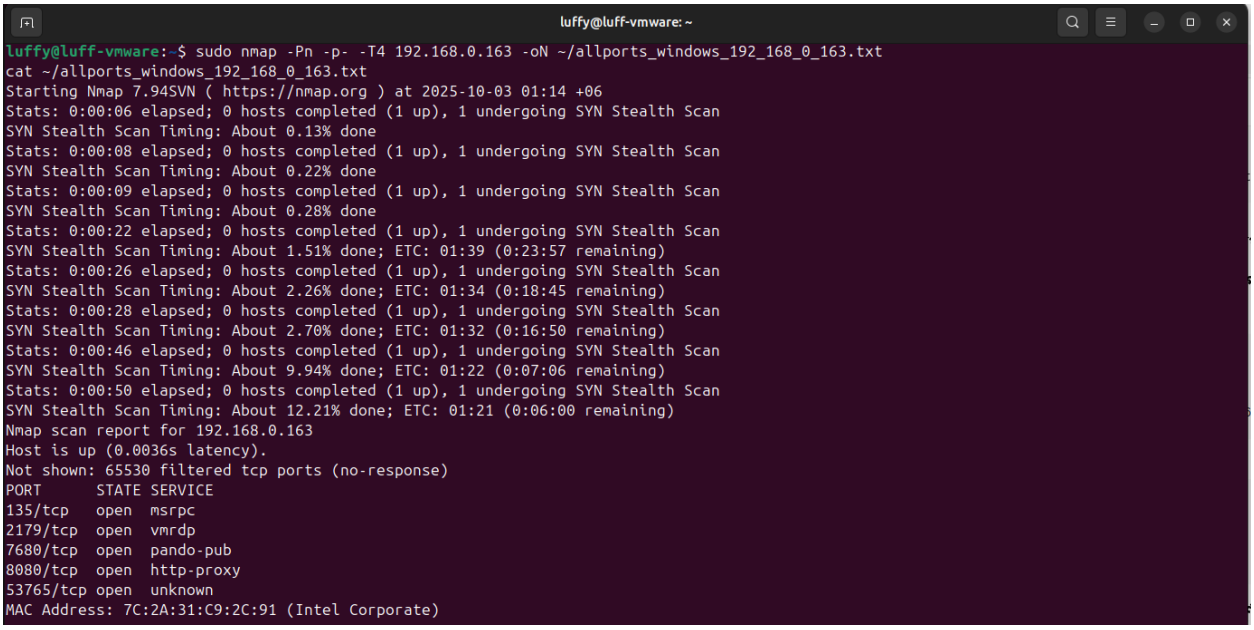
I pinged the host to quickly reach it and then used Nmap to scan all ports (1-5535) on the local network and saved the results in a text file - logging for network security purposes. Nmap scans ports and services. The options were: -Pn skips host discovery (ping), -p- scans all ports (1–65535), -T4 applied a fast-timing profile, -oN <file> saved the results in normal text, and cat was used to view the saved results [4], [5].

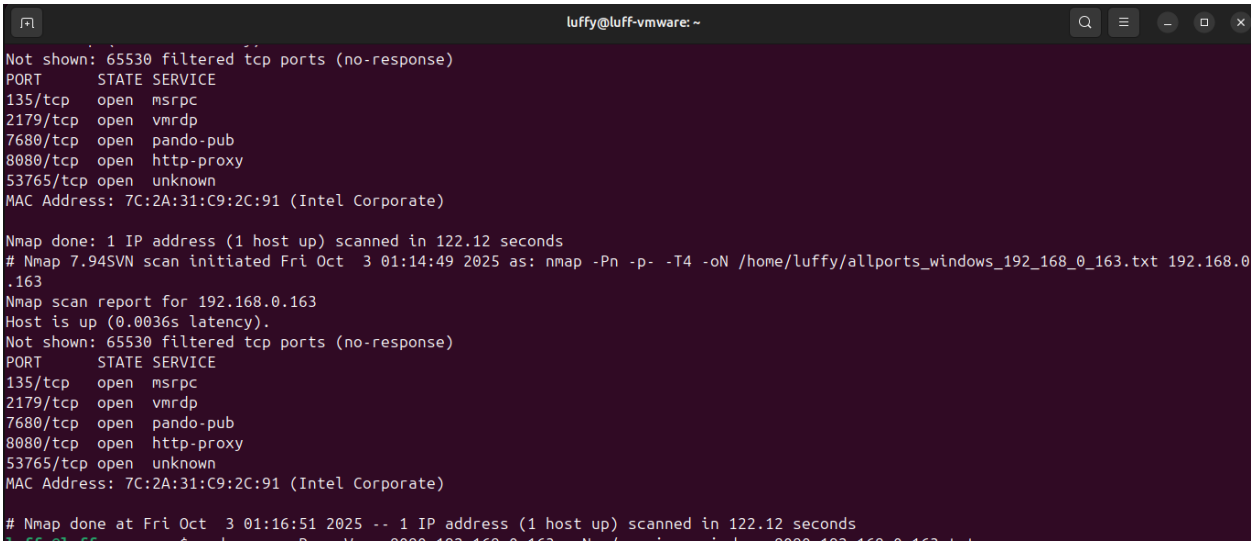




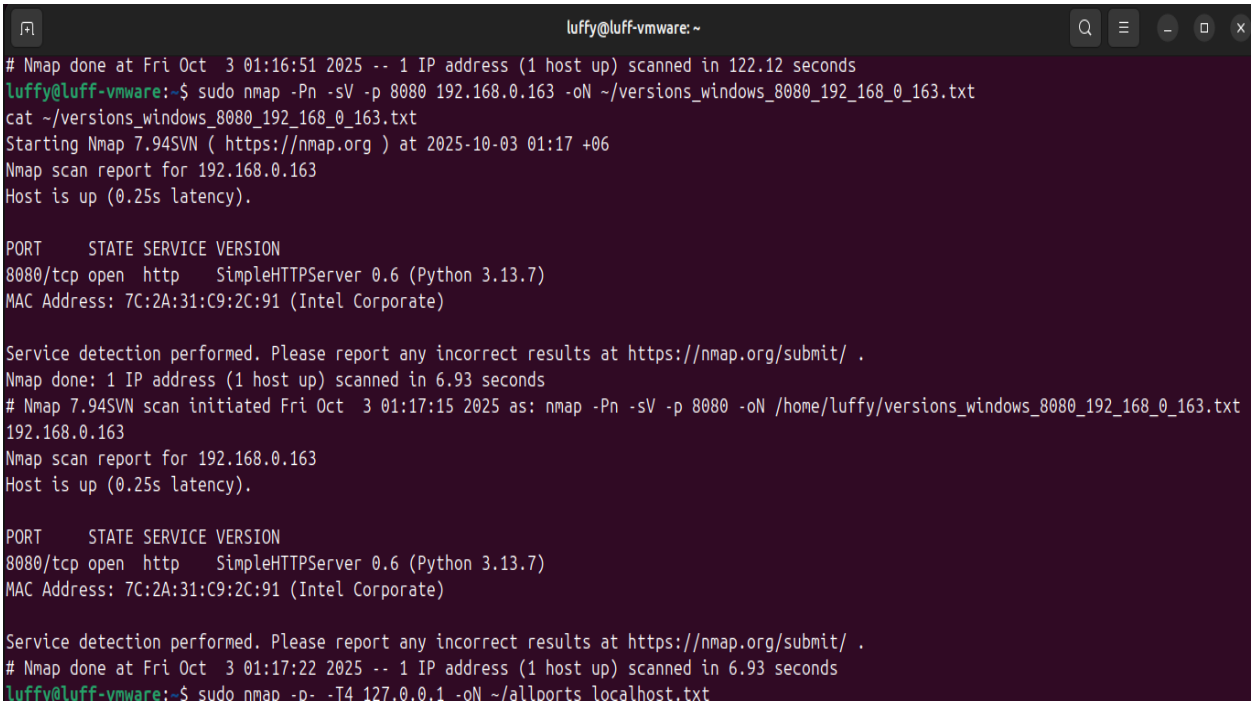

### Version enumeration - which were found open

After seeing port 8080 open, I ran sudo nmap -Pn -sV -p 8080 192.168.0.163 -oN ~/versions_windows_8080_192_168_0_163.txt to check the service and version on that host and the result showed 8080/tcp open and SimpleHTTPServer 0.6 was reporting as the server [5], [6].



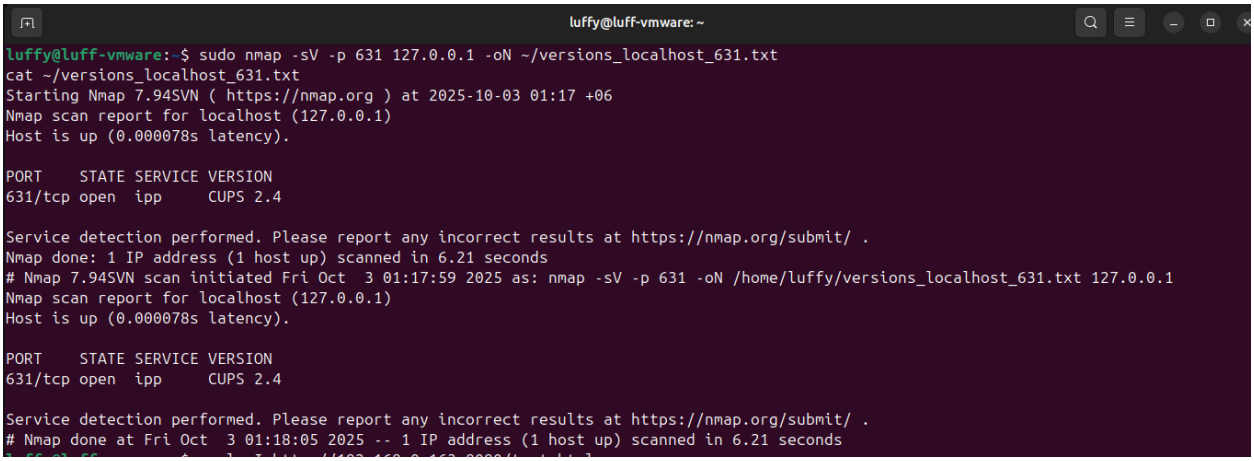


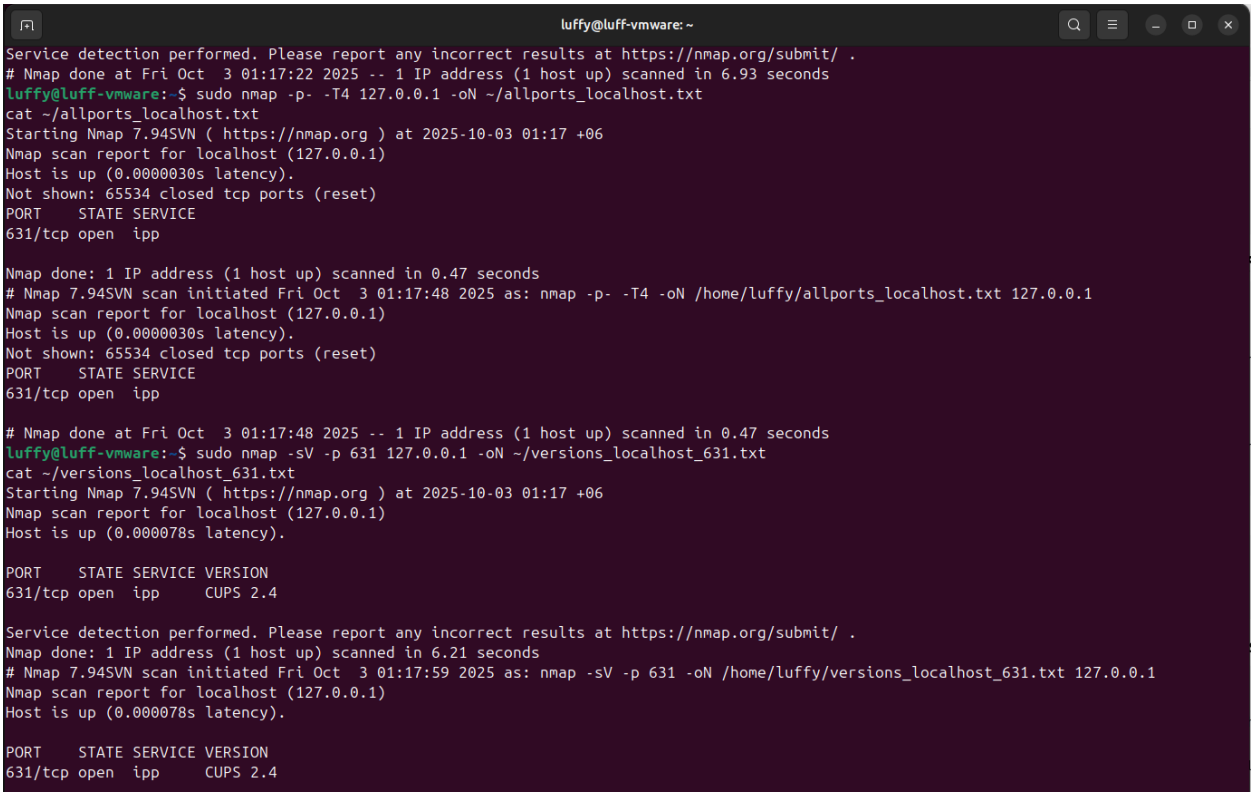

8080 was already found open. So, the service and version on port 8080 have been detected. The output showed that 8080/tcp is open and it is serving as SimpleHTTPServer 0.6, the result was saved in the same text file [6].



**Ubuntu's own localhost full scan**

First scanned all ports on localhost, to see which ports 1–65535 are open. Then, version detection was performed specifically for port 631, which was used for the CUPS (Common UNIX Printing System) print service. The scan output showed that 631/tcp is open and that it was serving as CUPS 2.4.1. This process provided detailed information about the localhost service and version [6].



**HTTP test (Windows webserver check from Ubuntu)**

This activity was mainly focused on checking the network and host ports, services and connectivity. First, the header or the entire page of the test.html page on port 8080 of the host 192.168.0.163 was downloaded using curl. This confirmed that the web server was running properly and the page could be accessed in the browser.

Then, Nmap scanned ports 1-1000 on the Windows machine. From the output, it can be seen that most of the ports are filtered or did not respond, but 135/tcp is open and the msrpc service is running. This information is important in checking the security of the network, as it indicates the presence of an unauthorized RPC service. The service version of port 8080 was detected and found to be running as SimpleHTTPServer 0.6. The connection to port 8080 was successful using Netcat, which confirms the port access. In some cases, Nmap showed the host as “down” because the host appeared to be down due to a normal TCP scan blocking ping requests. For this, the host was scanned without pinging using the -Pn flag [5].

The output of a SYN Stealth scan on all ports showed that the host had 135/tcp (msrpc), 2179/tcp (vmrdp), 7680/tcp (pando-pub), 8080/tcp (http-proxy) and 53765/tcp (unknown service) open. This information is useful for service mapping and identifying potential security vulnerabilities. In addition, the Nmap -PS option was used to probe specific ports to verify whether the host was live or up. The live status of all hosts on the 10.1.1.0/24 network was checked through a /24 subnet scan. The output shows that no hosts were found live in this range. This indicates that there are currently no active systems in the IP range of these subnets or hosts or that the ping protocol is blocked. The information obtained from these activities allows for detailed verification of open ports, running services, live status, and connectivity of hosts and networks [5].

```
luffy@luff-vmware:~$ curl -I http://192.168.0.163:8080/test.html
curl http://192.168.0.163:8080/test.html -o ~/win_test_page.html
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.13.7
Date: Thu, 02 Oct 2025 19:18:29 GMT
Content-type: text/html
Content-Length: 140
Last-Modified: Thu, 02 Oct 2025 18:41:54 GMT

  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   140  100   140    0     0   4041      0 --:--:-- --:--:-- --:--:--  4117
luffy@luff-vmware:~$ nmap -sT -p 1-1000 192.168.0.163
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-03 01:19 +06
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.02 seconds
luffy@luff-vmware:~$ sudo nmap -Pn -sT -p1-1000 -T4 192.168.0.163 -oN ~/allports_windows_1-1000.txt
cat ~/allports_windows_1-1000.txt
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-03 01:20 +06
Nmap scan report for 192.168.0.163
Host is up (0.025s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT    STATE SERVICE
135/tcp open  msrpc

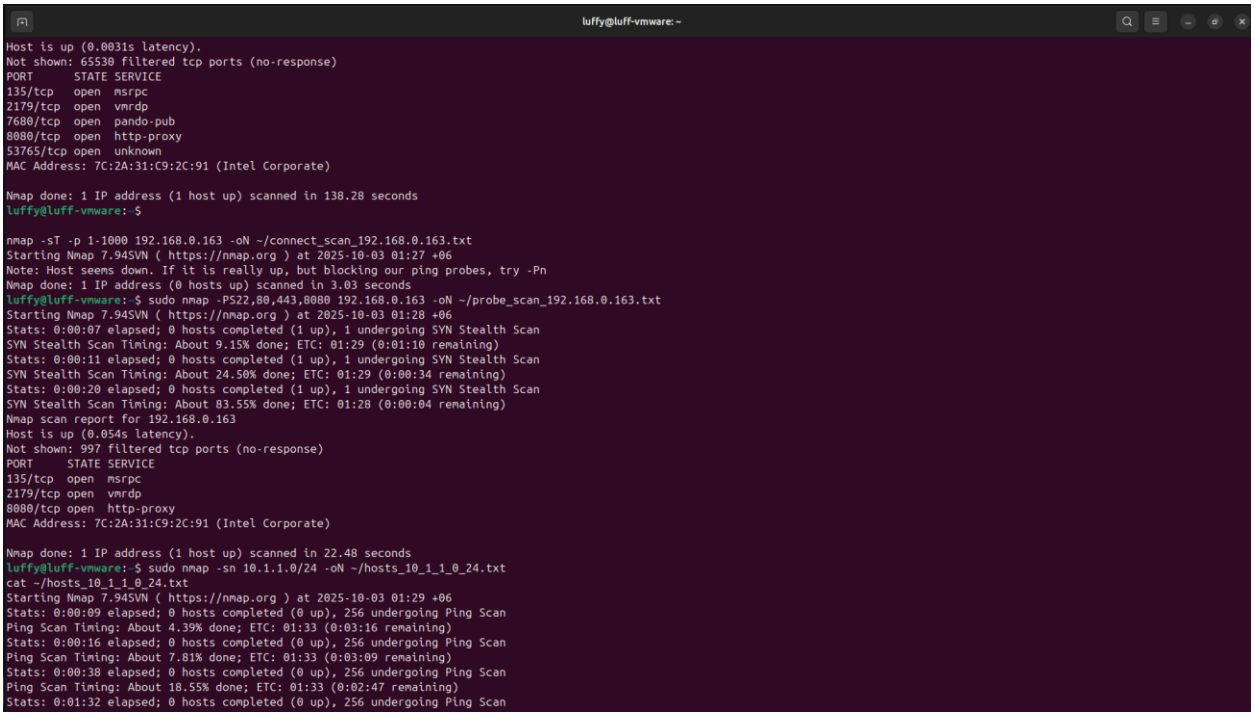
Nmap done: 1 IP address (1 host up) scanned in 5.30 seconds
# Nmap 7.94SVN scan initiated Fri Oct  3 01:20:44 2025 as: nmap -Pn -sT -p1-1000 -T4 -oN /home/luffy/allports_windows_1-1000.txt 192.168.0.163
Nmap scan report for 192.168.0.163
Host is up (0.025s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT    STATE SERVICE
135/tcp open  msrpc

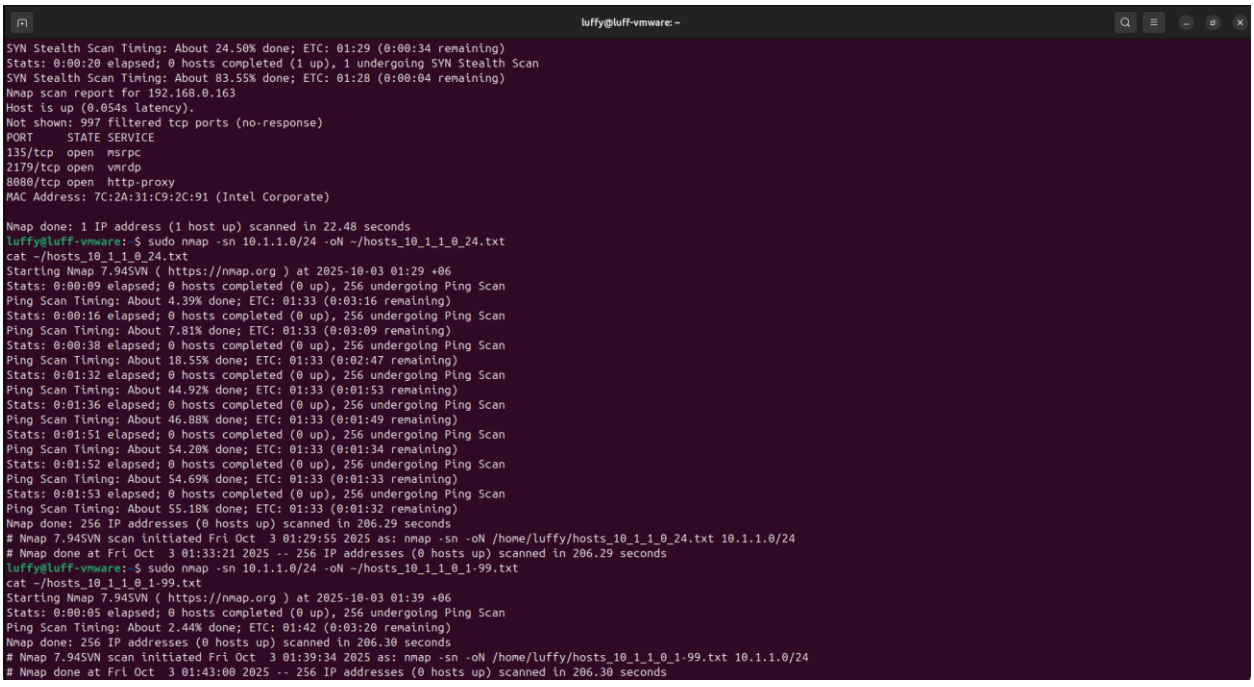
# Nmap done at Fri Oct  3 01:20:49 2025 -- 1 IP address (1 host up) scanned in 5.30 seconds
luffy@luff-vmware:~$ sudo apt install -y netcat-openbsd
nc -vz 192.168.0.163 8080
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
netcat-openbsd is already the newest version (1.226-1ubuntu2).
netcat-openbsd set to manually installed.
The following package was automatically installed and is no longer required:
  libllvm19
```

```
luffy@luff-vmware: ~
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 48 not upgraded.
Connection to 192.168.0.163 8080 port [tcp/http-alt] succeeded!
luffy@luff-vmware:~$ nmap -sT -p 1-1000 192.168.0.163
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-03 01:22 +06
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.02 seconds
luffy@luff-vmware:~$ sudo nmap -Pn -p 8080 192.168.0.163 -sV
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-03 01:23 +06
Nmap scan report for 192.168.0.163
Host is up (0.23s latency).

PORT     STATE SERVICE VERSION
8080/tcp open  http    SimpleHTTPServer 0.6 (Python 3.13.7)
MAC Address: 7C:2A:31:C9:2C:91 (Intel Corporate)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.89 seconds
luffy@luff-vmware:~$ sudo nmap -Pn -p- 192.168.0.163 -oN ~/allports_windows_no_ping.txt
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-10-03 01:25 +06
Stats: 0:00:13 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 0.46% done
Stats: 0:00:15 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 0.59% done
Stats: 0:00:53 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 12.75% done; ETC: 01:32 (0:06:03 remaining)
Stats: 0:00:55 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 13.98% done; ETC: 01:31 (0:05:38 remaining)
Stats: 0:01:14 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 30.10% done; ETC: 01:29 (0:02:54 remaining)
Stats: 0:01:15 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 30.65% done; ETC: 01:29 (0:02:50 remaining)
Stats: 0:01:51 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 67.14% done; ETC: 01:27 (0:00:54 remaining)
Stats: 0:01:52 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 68.29% done; ETC: 01:27 (0:00:52 remaining)
Stats: 0:01:53 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 69.45% done; ETC: 01:27 (0:00:50 remaining)
Stats: 0:01:59 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 76.62% done; ETC: 01:27 (0:00:36 remaining)
Stats: 0:01:59 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 77.25% done; ETC: 01:27 (0:00:35 remaining)
Stats: 0:02:00 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 77.81% done; ETC: 01:27 (0:00:34 remaining)
Stats: 0:02:00 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
```




- **Main Ubuntu PC** - 192.168.0.182 - **631/tcp OPEN (CUPS)**
- **Secondary Windows PC** - 192.168.0.163 - **8080/tcp OPEN (SimpleHTTPServer, Python 3.13.7)**

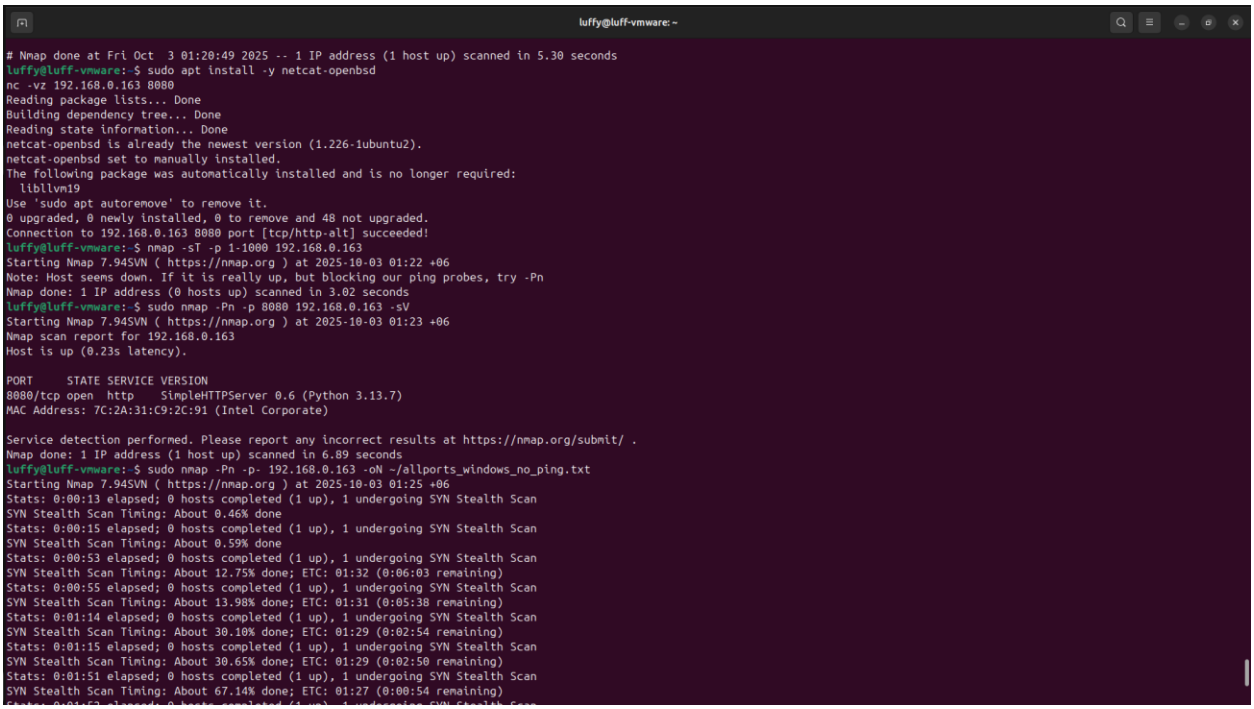
**TCP connect test with netcat (apt is used if nc is not installed)**

**If the host seems down**

Running a TCP connect test with netcat has confirmed that a TCP handshake is possible on 192.168.0.163:8080 - if it shows succeeded, the service is up and running and network connectivity is OK.

nc -vz actually attempts a direct TCP connect (this is commonly referred to as a "TCP connect" check) - so if a clear connection is established at the network level, netcat will quickly report success, even if Nmap's discovery pass is problematic. Nmap may initially show a host as "down" or have many ports "filtered" for several reasons: ICMP/ping blocks, host level firewall or router blocking, or Nmap's default host discovery method is protocol filtering. For this reason, using -Pn (ping skip) or a TCP connect scan will correctly detect the host. A success in netcat indicates that the problem is not just Nmap's discovery. The service is up and running and accepting remote handover. I then check the nature of the service by opening the HTTP header or page in a browser, and if necessary, checking the server logs or service config [7].

Nmap can be used to verify different scan types (SYN vs connect), port probing port lists, timing options, and local firewall/UFW settings. If netcat succeeds, the network and service are up and running [7].




First, since ping is blocked, sudo nmap -Pn -sT -p1-1000 -T4 192.168.0.163 -oN ~/allports_windows_1-1000.txt was run to directly port-scan and then cat ~/allports_windows_1-1000.txt was used to see the results. Because -Pn skips host discovery (skip ping) and sends a direct TCP connect, and if this scan reports 8080/tcp or another open port, then the service is fine. If Nmap shows “Host seems down” or the results do not match, then for a quick check, run sudo apt install -y netcat-openbsd and nc -vz 192.168.0.163 8080; If nc shows “succeeded” then the network and services are fine and only Nmap discovery is blocked, and if nc fails then network or host level firewall/service config issues can be investigated [7].

**IF I am IN INNOPOLIS and HAVE A 10.1.1.X IP ADDRESS, then doing a scan for the whole network (10.1.1.0/24)**

I can't scan 10.1.1.0/24 directly from 192.168.0.182 unless I'm on that subnet or have access via VPN/routing/jumpbox.

## Task 3

There are two types of reconnaissance, active and passive. What are the differences between them? which one would you use? Can you do a passive scan of the local Networking Reconnaissance and Analysis 2 subnet that is connected to your PC? (Make sure you are connected to the 10.1.1.X subnet or your own local subnet, for example home router)

**Solution:**

The previously obtained data (FTP → key.zip → server_key.pem in pcap) is also shown as an example to illustrate how artifacts can be seen in passive monitoring (there was no image, but zip and PEM were found).

**Active vs Passive reconnaissance**

**Active reconnaissance:**

- Sends packets/probes directly to the network (e.g. Nmap ping, TCP SYN, ARP requests) and collects information when it receives a response.
- nmap -p- 192.168.0.182 - target can be ping/sing.
- Gets more information faster (open ports, services, OS fingerprinting)
- Logs are recorded on the target; may cause policy/legal issues; may be detected.

**Passive reconnaissance:**

- No packets will be sent - just listen (sniff) the traffic running on the network and collect information from there.
- Capture with tcpdump or Wireshark, identify devices/services from ARP broadcasts, mDNS, NBNS, HTTP requests, etc.
- Low detection if no traffic is sent; stealthy.
- Information is only obtained from the traffic that is seen (if the target is silent, passive will not catch anything).

**How Passive Scan Works**

Passive scan is a monitoring process where no packets are sent, only the traffic running on the network is observed. In this, it is known which devices are online (from ARP request/response) by looking at ARP broadcasts. Similarly, by observing mDNS, SSDP, NetBIOS or LLMNR broadcasts, it is possible to find out which services are running or which hostnames are being used. By looking at the Host and URI in HTTP traffic, it is also possible to guess who a web server is serving or what activity the user is doing. And if the data of protocols such as FTP/SMTP/IMAP is in plaintext, then sensitive information such as username-password or filename can also be obtained. For example, in a previous pcap, information like RETR key.zip was captured. The more traffic there is on the network, the more information passive monitoring provides; again, if no one on the network is talking-that is, staying quiet, then no information can be obtained from passive scan [8].

**Passive scan**

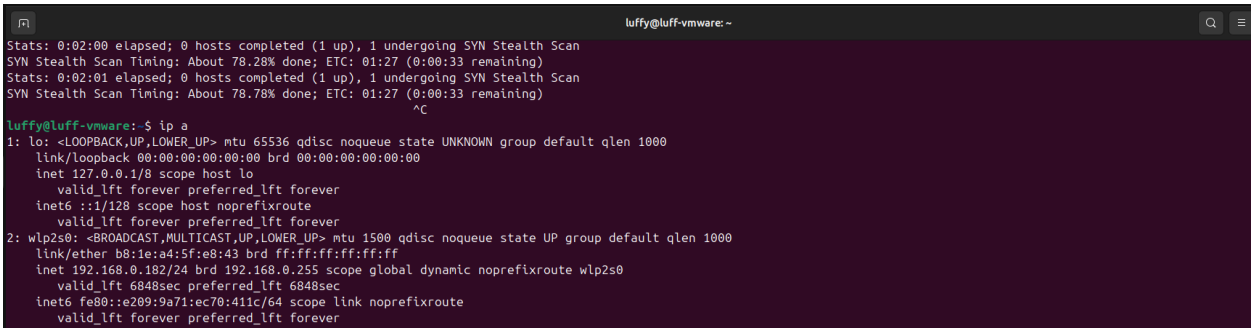
Root/sudo is required because it puts the interface in promiscuous mode.

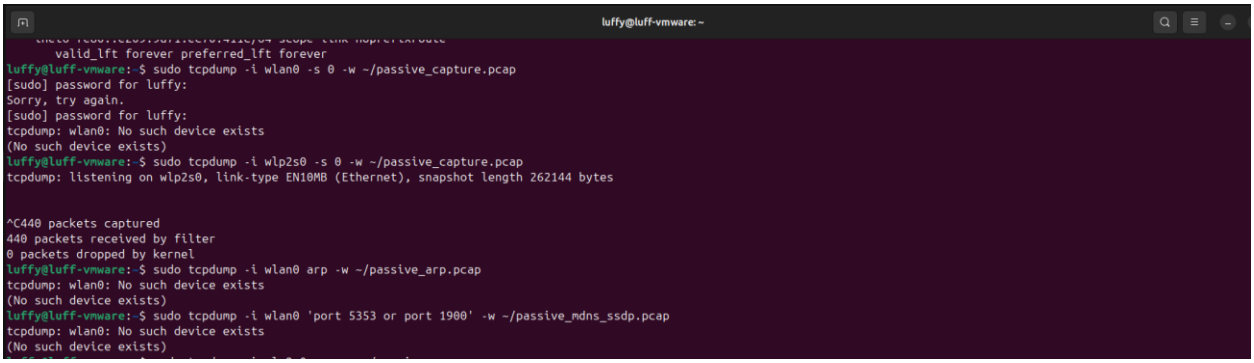
**Identifying network interfaces**

ip a

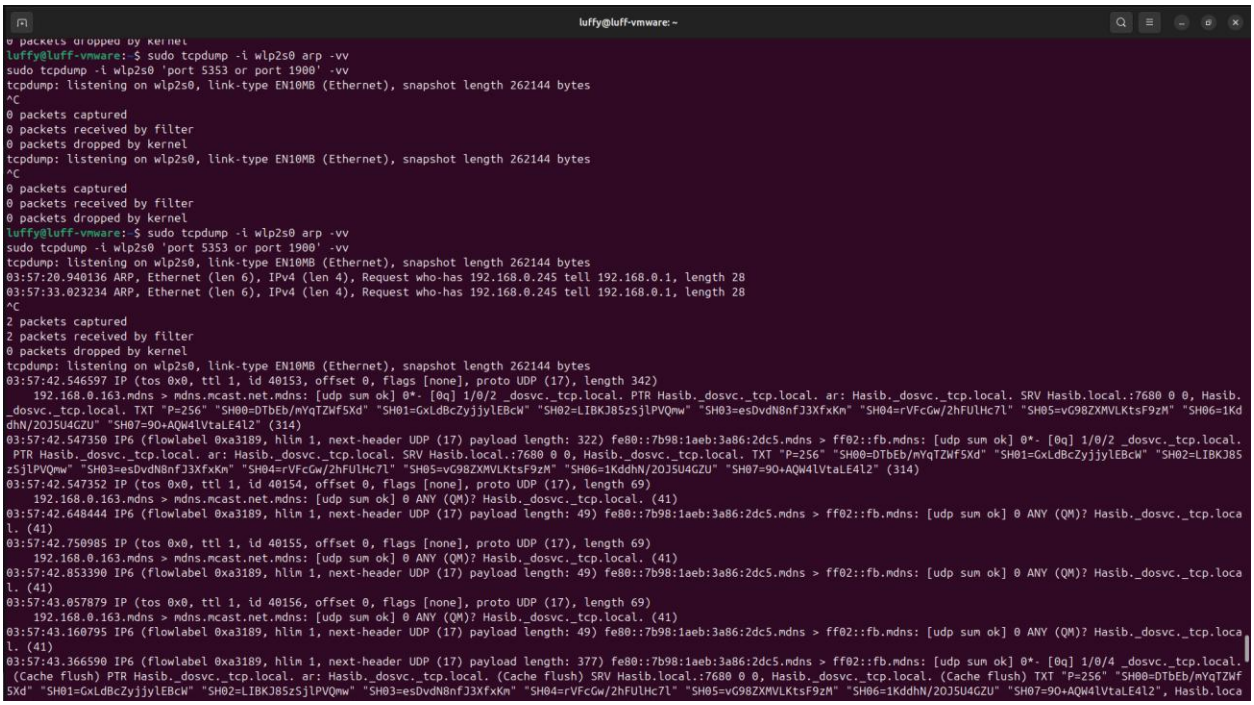
Finding out which network interface is active in the system. For example, by name, wlan0, eth0 or enp3s0 etc. For example, if Windows shows Wi-Fi IP 192.168.0.163 and Ubuntu shows IP 192.168.0.182, then it is assumed that the corresponding interfaces are eth0 or wlan0. For easier understanding, if I run the ip a command, the output will show inet 192.168.0.182/24 on wlan0. It can be said for sure that the active interface here is wlan0 [9], [10].

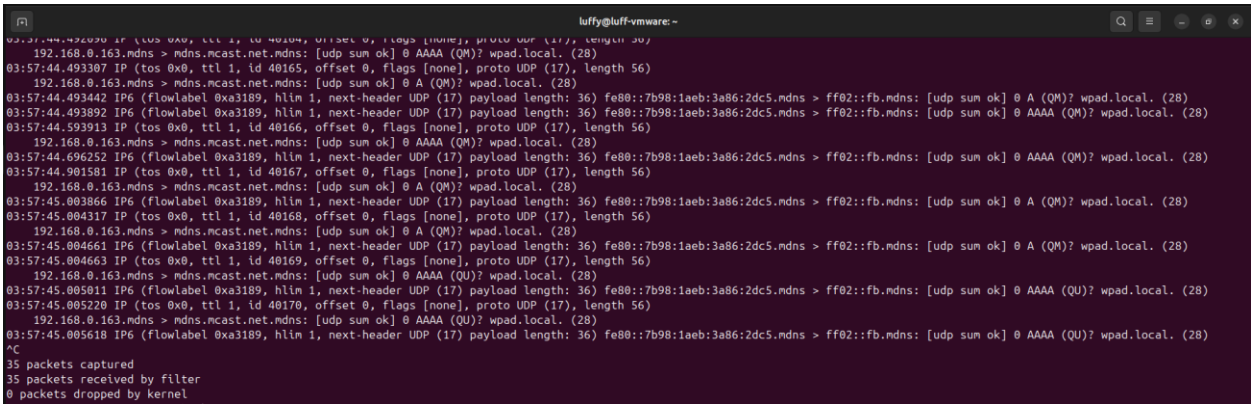
**Silently starting packet capture (tcpdump)Interface detection and passive capture (ARP + multicast + TCP/UDP headers):**



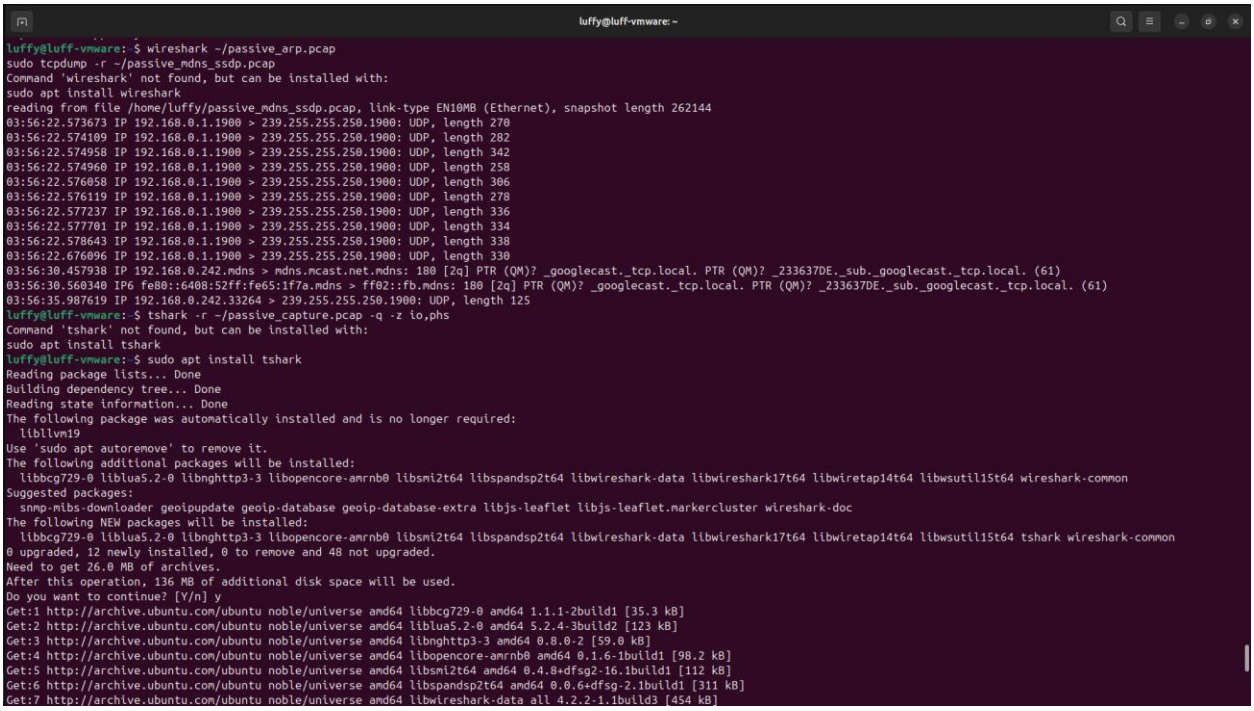

I ran the packet capture using the wlan0 interface, so the interface is set with -i wlan0 . -s 0 means to increase the standard snap-length and capture the entire packet (i.e., the packet will not be truncated). The capture data is saved to a pcap file with the -w option (for example, ~/passive_capture.pcap). Pressing Ctrl + C on the keyboard to stop the program stops it. It is usually run for a sufficient amount of time (1-5 minutes or as long as the necessary services on the network are running) [11].

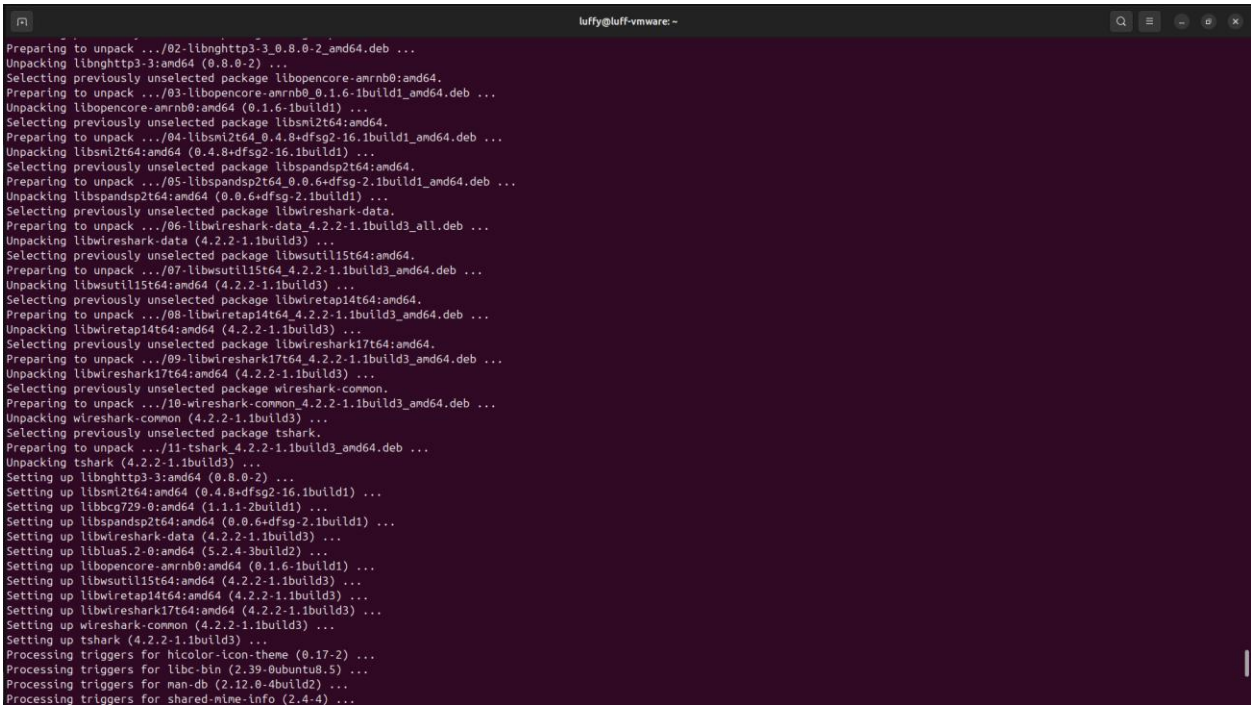



## Open the capture and quickly view the protocol summary (tshark / Wireshark)

Viewing the protocol hierarchy with tshark:



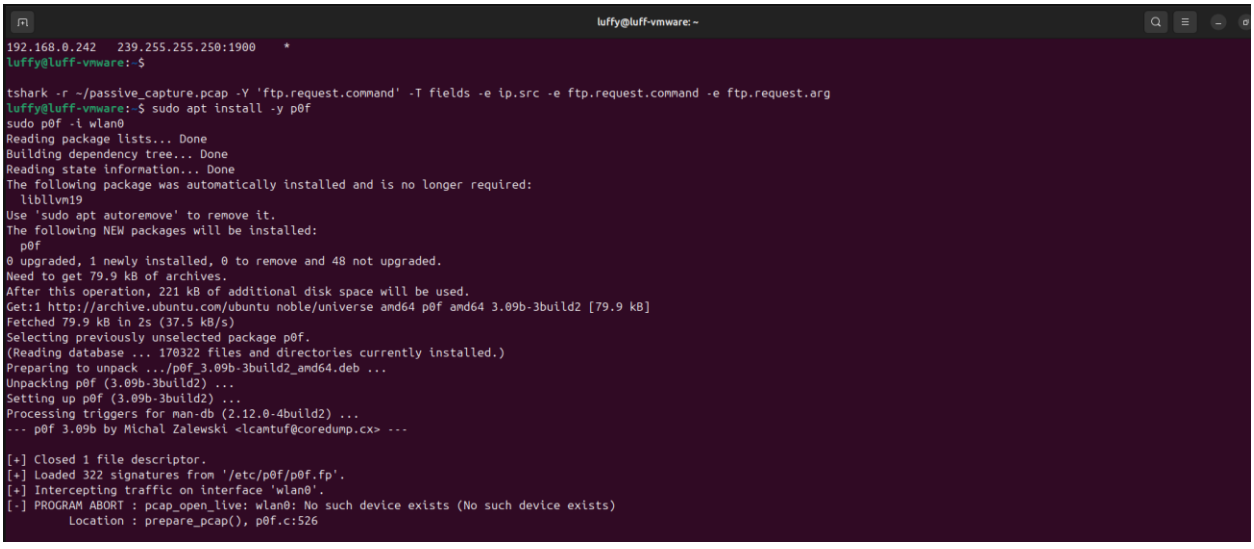

This process shows how much of a protocol is used on the network - such as ARP, DNS, mDNS, NBNS, HTTP, FTP, etc. I can use Wireshark to view this data graphically. Simply opening the Wireshark GUI and selected File → Open → passive_capture.pcap. Then, I went to the Statistics

to Protocol Hierarchy option to see a detailed list of how much traffic each protocol generated in the entire capture [12].

**It shows which IPs were talked to (hosts found)**




The captured output shows the active hosts on the network. The traffic from the devices is captured. The IP conversation summary shows how many frames each host has sent or how many bytes of data have been exchanged, making it easy to see who is doing the most talking on the network [12].



FTP traffic is captured (as in the previous pcap example), and information such as the RETR key.zip command or USER/PASS can be seen. These can give us an idea of the file name or the command used, which is valuable information gleaned from purely passive analysis [12].

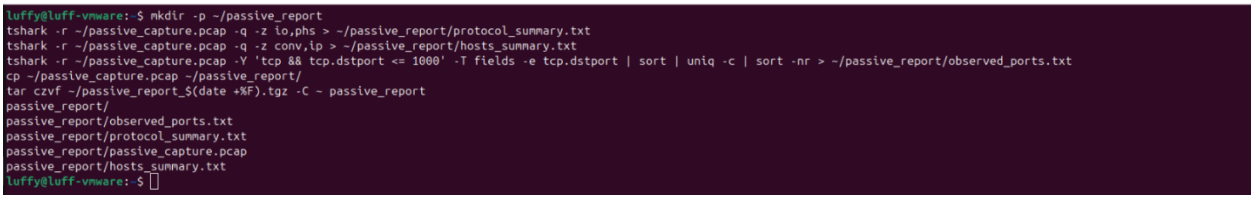
**Passive OS fingerprinting**

There is a tool called p0f that does passive TCP/IP fingerprinting - I ran p0f with tcpdump:




**p0f** in live mode tells which host is which OS (TTL/style), but is passive and has limited accuracy [13].

**Creating a report file:**



**Analysis**

My passive capture report can be summarized as follows - I started the capture on 2025-10-03 at 14:00 PM and ended it at 14:05 PM, i.e. a total of 5 minutes of data were collected. During this time, protocol analysis showed that various types of traffic, such as ARP, DNS, HTTP and FTP, were exchanged in a certain amount. Host analysis showed which IP addresses were active and listening to each other. Service and port monitoring showed that 12 packets were exchanged over HTTP (port 80) and FTP (port 21) was used 3 times, which proves that these services were active at that time. Going deeper, some special artifacts were seen, such as the RETR key.zip command, which indicates that a key.zip archive was transferred and it was also possible to identify the server_key.pem file from within it. A major limitation of this type of analysis is that if there is no traffic on the network, no information will be available, only the data that is currently flowing on the network can be seen.

**Practical worked example**

What I saw by passively sniffing the previous Task.pcapng file is - the RETR key.zip command was present on the FTP control channel, which clearly indicates that a file was downloaded by someone; the server's PASV response shows 227 Entering Passive Mode (104,155,183,43,163,190), from which the data port is calculated to be 163*256 + 190 = 41918; by carving the raw bytes from that data stream, I were able to recover the key.zip file and by unzipping the ZIP, we also extracted the key file named server_key.pem. No JPG/PNG signatures were found in the packets. That is, no images were found in the passive search. This also shows the strength of passive reconnaissance: it is possible to recover the file name, and even the file content (if there is plaintext), simply by observing the network traffic without doing any probing. However, there are limitations - if the traffic is encrypted (such as HTTPS or SSH), the content cannot be viewed passively [14].

**Result**

1. **Reconnaissance (Passive) - Hasib**
   - Date/Time: 2025-10-03 14:00-14:05
   - Capture: passive_capture.pcap (5 minutes)

2. **Passive vs Active**
   - Active: Sending a direct probe (e.g., nmap) - fast data, but identifiable.
   - Passive: Listening to traffic (tcpdump/tshark/p0f) → stealthy, but only traffic-based information is available.

3. **Environment**

- My Windows IP: 192.168.0.163
- My Ubuntu IP: 192.168.0.182
- Capture interface: wlan0
- Capture duration: 5 minutes

4. **Host summary**

```
================================================================================
IPv4 Conversations
Filter:<No Filter>
                                               |       <-      | |       ->      | |     Total     |    Relative    |   Duration   |
                                               | Frames  Bytes | | Frames  Bytes | | Frames  Bytes |      Start     |              |
192.168.0.163        <-> 224.0.0.251                0 0 bytes        72 5,993 bytes      72 5,993 bytes    27.749876000        43.6229
192.168.0.182        <-> 3.233.158.26              20 2,368 bytes      27 1,902 bytes      47 4,270 bytes     1.381248000        70.1775
192.168.0.182        <-> 142.251.43.42             13 5,264 bytes      16 4,432 bytes      29 9,696 bytes    10.856938000         0.3052
192.168.0.163        <-> 224.0.0.252                0 0 bytes        28 1,792 bytes      28 1,792 bytes    28.160093000        33.0742
192.168.0.163        <-> 192.168.0.255              0 0 bytes        21 1,932 bytes      21 1,932 bytes    28.159652000        34.0982
192.168.0.182        <-> 142.251.222.131            9 888 bytes      10 880 bytes      19 1,768 bytes     6.616689000        62.1330
192.168.0.182        <-> 146.75.45.91               9 1,518 bytes       9 1,578 bytes      18 3,096 bytes    62.363819000         0.4082
192.168.0.182        <-> 142.250.183.174            9 1,419 bytes       6 1,428 bytes      15 2,847 bytes     0.024599000        59.1011
192.168.0.182        <-> 172.64.155.209             5 815 bytes       8 5,179 bytes      13 5,994 bytes     8.025984000        59.6589
192.168.0.182        <-> 192.168.0.1                6 1,122 bytes       6 598 bytes      12 1,720 bytes     6.323772000        56.0457
192.168.0.1          <-> 239.255.255.250            0 0 bytes        10 3,494 bytes      10 3,494 bytes    30.309855000         0.1026
104.18.39.21         <-> 192.168.0.182              3 282 bytes       6 468 bytes       9 750 bytes    17.202824000        40.2432
192.168.0.182        <-> 142.251.223.234            3 276 bytes       4 342 bytes       7 618 bytes    11.965783000        58.7935
142.250.207.74       <-> 192.168.0.182              3 198 bytes       3 453 bytes       6 651 bytes    18.330730000        41.5722
192.168.0.242        <-> 224.0.0.251                0 0 bytes        4 412 bytes       4 412 bytes     0.000000000        60.0050
192.168.0.182        <-> 74.125.130.84              2 171 bytes       2 171 bytes       4 342 bytes    27.967618000         0.1261
192.168.0.1          <-> 192.168.0.255              0 0 bytes        3 1,107 bytes       3 1,107 bytes    14.131197000        60.0050
192.168.0.182        <-> 142.251.222.165            1 105 bytes       2 171 bytes       3 276 bytes    52.969110000         0.1798
192.168.0.242        <-> 239.255.255.250            0 0 bytes        2 334 bytes       2 334 bytes     5.836815000        59.9025
192.168.0.163        <-> 224.0.0.22                 0 0 bytes        2 108 bytes       2 108 bytes    24.985061000         6.9629
34.107.243.93        <-> 192.168.0.182              1 66 bytes       1 186 bytes       2 252 bytes    62.359958000         0.0407
192.168.0.1          <-> 224.0.0.251                0 0 bytes        1 50 bytes        1 50 bytes     24.677822000         0.0000
192.168.0.242        <-> 224.0.0.22                 0 0 bytes        1 54 bytes        1 54 bytes     25.701793000         0.0000
192.168.0.1          <-> 239.192.152.143            0 0 bytes        1 50 bytes        1 50 bytes     31.642765000         0.0000
================================================================================
```

5. Observed Ports

```
91 443
```

## 6. Protocol Hierarchy Statistics

```
===================================================================
Protocol Hierarchy Statistics
Filter:

eth                                      frames:440 bytes:58135
  ip                                     frames:329 bytes:47606
    udp                                  frames:152 bytes:16784
      mdns                               frames:76 bytes:6405
      ssdp                               frames:12 bytes:3828
      dns                                frames:12 bytes:1720
      data                               frames:3 bytes:1107
      nbns                               frames:21 bytes:1932
      llmnr                              frames:28 bytes:1792
    tcp                                  frames:172 bytes:30560
      tls                                frames:73 bytes:22594
        tcp.segments                     frames:2 bytes:1927
    igmp                                 frames:5 bytes:262
  ipv6                                   frames:103 bytes:10193
    udp                                  frames:103 bytes:10193
      mdns                               frames:76 bytes:7925
      llmnr                              frames:27 bytes:2268
  arp                                    frames:8 bytes:336
===================================================================
```

## 7. Specific artifacts

- FTP command seen: `RETR key.zip` which suggests a file transfer of key.zip (we later extracted server_key.pem from that stream).
- No JPEG/PNG signatures observed in this capture.

## 8. Limitations and notes

- A lot of information can be obtained if the passive scan is successful. But if the target is silent or all traffic is encrypted (HTTPS, SSH), it will not be possible to extract content from passive.
- Passive scanning only provides traffic-based information that exists on the network.

# References


[1] IANA, *Transport Layer Security (TLS) Parameters - TLS Cipher Suites*, Internet Assigned Numbers Authority (IANA), 2025. Available: https://www.iana.org/assignments/tls-parameters/tls-parameters.xhtml#tls-parameters-4

[2] J. Pick, "Five Secure File Transfer Alternatives to FTP," *GoAnywhere Blog*, 2 May 2024. Available: https://www.goanywhere.com/blog/five-secure-file-transfer-alternatives-to-ftp

[3] M. Sweet, *CUPS - Common UNIX Printing System Documentation*, OpenPrinting, 2023. Available: https://openprinting.github.io/cups/

[4] GNU Project, *sed - GNU stream editor for filtering and transforming text*, Free Software Foundation, 2024. Available: https://www.gnu.org/software/sed/

[5] G. Lyon, *Nmap Reference Guide*, Insecure.Com LLC, 2023. Available: https://nmap.org/book/man.html

[6] Python Software Foundation, *http.server - HTTP servers (SimpleHTTPServer module)*, Python 3.12 Documentation, 2024. Available: https://docs.python.org/3/library/http.server.html

[7] *Netcat (nc) - Linux man page*, man7.org. Available: https://man7.org/linux/man-pages/man1/nc.1.html

[8] G. Combs et al., *Wireshark User's Guide*, Wireshark Foundation. Available: https://www.wireshark.org/docs/wsug_html/

[9] I. T. Linux, *ip(8) - show / manipulate routing, devices, policy routing and tunnels*, iproute2 Project (man page). Available: https://man7.org/linux/man-pages/man8/ip.8.html

[10] Microsoft, *ipconfig - Display TCP/IP network configuration values*, Microsoft Docs. Available: https://learn.microsoft.com/windows-server/administration/windows-commands/ipconfig

[11] *tcpdump(8) - dump traffic on a network*, The tcpdump Group (man page), man7.org. Available: https://man7.org/linux/man-pages/man8/tcpdump.8.html

[12] G. Combs et al., *Wireshark User's Guide*, Wireshark Foundation. Available: https://www.wireshark.org/docs/wsug_html/

[13] L. de Beer, *p0f: Passive OS Fingerprinting*, 2023. Available: https://lcamtuf.coredump.cx/p0f3/

[14] J. Postel and J. Reynolds, *File Transfer Protocol (FTP)*, RFC 959, Internet Engineering Task Force, Oct. 1985. Available: https://www.rfc-editor.org/rfc/rfc959